INFIRMO

Le réseau professionnel des infirmiers libéraux

CAHIER DES CHARGES

Projet T-ESP-800 - Epitech Nancy

Tetiana Lombardi · Maxence Noizet · Yohan Baechlé · Louis Chenot

Pierre-Arthur Toutouom Yountebo · Ziad Tiarimti · Lazaro Marrero Rousse

Table des matières

Table des matières **1**
1. Présentation du projet **4**
1.1. Contexte 4
1.2. Objectifs 4
1.2.1. Objectif principal 4
1.2.2. Objectifs secondaires 4
1.3. Périmètre du projet 4
1.3.1. Public cible 4
1.3.2. Plateforme 5
1.3.3. Limites actuelles 5
2. Analyse de l'existant et problématique **5**
2.1. Situation actuelle 5
2.2. Conséquences 5
2.3. Pourquoi ce marché existe 6
3. Fonctionnalités attendues **6**
3.1. Administration des utilisateurs 6
3.2. Correspondance et recherche 7
3.3. Gestion des remplacements 7
3.4. Communication sécurisée 8
3.5. Intelligence artificielle 8
3.6. Sécurité et conformité 9
3.7. User Stories 10
4. PBS (Product Breakdown Structure) **12**
5. WBS (Work Breakdown Structure) **14**
1. Gestion des utilisateurs 14
2. Recherche, matching et calendrier 14
3. Gestion des remplacements et missions 15
4. Communication et transmissions 16
5. Sécurité, RGPD et conformité HDS 16
6. Architecture applicative 17
7. Infrastructure, DevOps et déploiement 18
8. Tests, qualité et validation 18
9. Documentation, organisation et promotion 19
Description des fonctions du WBS 20
6. OBS (Organizational Breakdown Structure) **21**
6.1. Composition de l'équipe 22
6.2. Matrice de responsabilités (RACI) 22
7. Backlog **25**

7.1. Objectif de la section....25
7.2. Hypothèses de charge et de réunions....26
7.3. Traçabilité WBS....26
7.4. Backlog général consolidé....28
7.5. Lecture du backlog....31
7.6. Synthèse de conformité au cadrage projet....31
8. Compétences de l'équipe et justification du recrutement....32
8.1. Compétences identifiées nécessaires....32
8.2. Justification professionnelle du recrutement....33
9. Méthodologie de projet....34
9.1. Choix de la méthodologie: ScrumBan....34
9.2. Rituels Scrumban....34
9.3. Outil de planification et de gestion....34
9.4. Roadmap et planning macro....35
10. Choix technologiques et architecture....36
10.1. Architecture globale....36
10.2. Modèle de données (ERD)....38
10.3. Stack technique....41
10.4. Stratégie de tests....41
10.5. Indicateurs de performance (KPI)....43
11. Plan de qualité....43
11.1. Processus de développement....43
11.2. Normes et bonnes pratiques....44
11.3. Conventions Git....44
11.4. Gestion de la configuration....44
11.5. Gestion des tickets....45
11.6. Gestion des livrables et déploiements....45
11.7. Rôles et prérogatives....45
11.8. Onboarding et ramp-up....45
12. Definition of Done (DoD)....45
13. Analyse des risques....46
14. Ressources matérielles et estimation des coûts....48
15. Contraintes techniques et réglementaires....49
15.1. Architecture....49
15.2. Sécurité et conformité....49
15.3. Performance....49
15.4. Données patients (RGPD)....49
15.5. Comment la conformité RGPD sera vérifiée....50

16. Livrables attendus..... 51
17. Évolutions possibles..... 51
18. Glossaire..... 52

1. Présentation du projet

1.1. Contexte

En 2026, on dénombre plus de 100 000 infirmiers en activité libérale en France. Chaque structure est juridiquement obligée de garantir la continuité des soins, ce qui crée un besoin permanent et structurel en professionnels de remplacement qualifiés. Ces experts opèrent dans un environnement de mobilité incessante et de restrictions réglementaires strictes (RGPD, secret médical), ce qui limite l'utilisation d'outils compliqués ou peu adaptés. Néanmoins, il n'y a pas actuellement de plateforme professionnelle spécialisée pour répondre à ces exigences particulières. Infirmo trouve sa place dans ce vide.

1.2. Objectifs

1.2.1. Objectif principal

- Concevoir une application mobile sécurisée qui facilite la mise en relation des infirmiers libéraux (remplacements, collaborations).
- Diffuser des données professionnelles de façon confidentielle et en accord avec le RGPD.
- Organiser les horaires, les demandes de remplacement et les itinéraires.
- Consolider les informations relatives aux soins et à l'activité de tous les jours.

1.2.2. Objectifs secondaires

- Minimiser les communications informelles non sécurisées (SMS, messageries publiques).
- Améliorer la rapidité de réponse en cas de nécessité de remplacement.
- Organiser les communications professionnelles parmi les professionnels de la santé.
- Suggérer une base adaptable qui comprend des fonctionnalités de pointe telles que l'intelligence artificielle, les recommandations et les automatisations.

1.3. Périmètre du projet

1.3.1. Public cible

- **Infirmiers** indépendants titulaires ayant un cabinet et à la recherche de remplaçants occasionnels ou réguliers.

- **infirmiers remplaçants** travaillant dans des cabinets ou sur des missions temporaires, recherchant des tournées qui leur conviennent.

1.3.2. Plateforme

Le projet met un accent particulier sur l'application mobile. Les infirmiers en pratique indépendante sont constamment en déplacement et ont besoin d'un outil qu'ils peuvent utiliser sur leur téléphone portable. À ce stade, l'idée de développer une version web n'est pas envisagée, car elle est considérée comme peu pertinente pour une utilisation sur le terrain.

1.3.3. Limites actuelles

- Le projet ne vise pas à remplacer un logiciel médical certifié.
- Pas de téléconsultation médicale intégrée.
- L'IA n'a pas vocation à prendre des décisions médicales, uniquement à assister le professionnel.

2. Analyse de l'existant et problématique

2.1. Situation actuelle

Sans outil spécifique, les infirmiers indépendants utilisent des méthodes non officielles pour coordonner leurs remplacements et échanges professionnels :

- **Groupes Facebook** : manquent de sécurité, encombrés de contenu inapproprié, sans contrôle ni possibilité de suivre les contributions.
- **Transmissions informelles** : lentes, restreintes spatialement et dépourvues de structure.
- **Annonces en ligne** : disséminées sur des sites généralistes, sans aucune centralisation ni suivi.
- **WhatsApp et les messages vocaux** : des informations de santé sensibles circulent sur des plateformes non conformes au RGPD, dépourvues d'un chiffrement approprié et d'une traçabilité adéquate, mettant ainsi en danger la responsabilité légale des professionnels.

2.2. Conséquences

Ces méthodes entraînent des problèmes tangibles et quantifiables :

- Les cabinets gaspillent de nombreuses heures à chercher un substitut compétent et disponible.
- Il est difficile pour les remplaçants de trouver des tournées qui correspondent à la fois géographiquement et professionnellement.

- L'envoi de données des patients expose les professionnels à des risques de non-conformité au RGPD.
- Aucune solution n'offre d'automatisation, de tri ou de gestion structurée des requêtes.

2.3. Pourquoi ce marché existe

La nécessité de remplaçants est constante et organisée : les congés, les maladies, les formations, ainsi que les pics d'activité ou l'accroissement du nombre de patients imposent à chaque cabinet de garantir la continuité des soins sans exception. Ce besoin persistera et aucune solution professionnelle actuelle ne le satisfait. Infirmo se situe exactement sur ce marché de niche.

3. Fonctionnalités attendues

Cette partie présente toutes les fonctionnalités du produit, réparties en modules fonctionnels. Chaque caractéristique est liée à des critères spécifiques pour sa mise en œuvre.

3.1. Administration des utilisateurs

Système complet de gestion des comptes utilisateurs, facilitant l'établissement de profils professionnels approfondis, l'authentification en toute sécurité et la gestion de documents juridiques.

ID	Fonctionnalité	Exigences	Priorité
F1.1	**Création de compte sécurisé**	Inscription avec numéro Ordre Infirmiers (validation auto), complétion du profil (zones, disponibilités, documents).	Haute
F1.2	**Upload documents légaux**	Formulaire mobile avec upload carte pro / RIB / assurance (PDF max 5 Mo).	Haute
F1.3	**Vérification Ordre Infirmiers**	Vérification du numéro Ordre via API publique (mock en dev).	Haute
F1.4	**Authentification 2FA**	Email/SMS 2FA optionnel pour renforcer la sécurité.	Moyenne

F1.5	**Profil professionnel**	Coordonnées, zone géographique, compétences, disponibilités, préférences.	Haute
F1.6	**Gestion rôles**	Distinction cabinet / remplaçant avec fonctionnalités adaptées.	Haute

3.2. Correspondance et recherche

Un algorithme propose automatiquement les trajets qui correspondent aux critères de l'utilisateur : distance, horaires, type de soins, chiffre d'affaires, disponibilité.

ID	Fonctionnalité	Exigences	Priorité
F2.1	**Filtres de recherche**	Sliders (distance < 50 km, CA > X €), checkboxes (soins : pansements, injections, etc.).	Haute
F2.2	**Algorithme de matching**	Tri par pertinence (distance + matching disponibilités + CA estimé).	Haute
F2.3	**Géolocalisation**	Carte géo avec pins tournées (Flutter + Google Maps).	Haute
F2.4	**Calendrier & disponibilités**	Visualisation rapide des besoins et des offres.	Moyenne
F2.5	**Notifications & alertes**	Notifications push lors de nouvelles tournées correspondant au profil.	Moyenne

3.3. Gestion des remplacements

Système essentiel qui offre la possibilité aux cabinets de publier des tournées et aux remplaçants de soumettre leurs candidatures. Processus complet de la présentation jusqu'à l'approbation.

ID	Fonctionnalité	Exigences	Priorité

F3.1	**Publication de tournées**	Cabinets publient des offres avec détails (dates, soins, CA, localisation).	Haute
F3.2	**Consultation des offres**	Remplaçants visualisent les tournées disponibles avec filtres.	Haute
F3.3	**Candidature & acceptation**	Workflow de candidature, proposition et acceptation.	Haute
F3.4	**Statuts des missions**	Suivi en temps réel de l'état des missions.	Moyenne
F3.5	**Historique & statistiques**	Historique des remplacements et statistiques d'activité.	Basse

3.4. Communication sécurisée

Un système de communication interne chiffré qui facilite le partage sécurisé de textes, d'images et de messages audio. Les enregistrements vocaux sont automatiquement transcrits et catégorisés.

ID	Fonctionnalité	Exigences	Priorité
F4.1	**Messagerie texte/photo**	Messagerie chiffrée de bout en bout entre professionnels.	Haute
F4.2	**Messages vocaux**	Enregistrement et envoi de vocaux avec transcription auto.	Haute
F4.3	**Transmissions sécurisées**	Partage d'informations patient dans un espace dédié et sécurisé.	Haute

3.5. Intelligence artificielle

L'intelligence artificielle est implantée en tant que support professionnel, logé sur un serveur sécurisé. Elle n'est pas destinée à prendre des décisions dans le domaine médical.

ID	Fonctionnalité	Exigences	Priorité
F5.1	**Transcription vocale**	Transcription automatique via Whisper (modèle local) des messages vocaux en texte.	Haute
F5.2	**Extraction d'informations**	Extraction et structuration des données issues des vocaux (patients, actes, observations).	Haute
F5.3	**Alimentation fiches patients**	Remplissage automatique des fiches patients à partir des données transcrites, avec validation manuelle.	Moyenne

3.6. Sécurité et conformité

Garantir la protection des données de santé sensibles tout au long du cycle de vie de l'application.

ID	Fonctionnalité	Exigences	Priorité
F6.1	**Auth forte**	Authentification Ordre Infirmiers + JWT, tokens renouvelables.	Haute
F6.2	**Chiffrement**	Chiffrement des données de santé en transit (TLS) et au repos (AES-256).	Haute
F6.3	**Hébergement HDS**	Hébergement conforme HDS (Hébergeur de Données de Santé).	Haute
F6.4	**Conformité RGPD**	Consentement explicite, durée conservation 30j, droit à l'oubli, anonymisation.	Haute
F6.5	**Audit & logs**	Traçabilité complète des accès et modifications.	Moyenne

3.7. User Stories

Les user stories ci-dessous constituent le backlog initial du projet et alimentent directement l'outil de gestion GitHub Projects..

ID	En tant que	Je veux	Afin de	Priorité
US-01	**Infirmier remplaçant**	créer mon profil avec mon numéro Ordre et mes disponibilités	être visible par les cabinets qui cherchent un remplaçant	Haute
US-02	**Cabinet titulaire**	publier une offre de remplacement avec les détails de la tournée	trouver rapidement un remplaçant qualifié	Haute
US-03	**Infirmier remplaçant**	rechercher des tournées par zone, dates et type de soins	trouver une mission adaptée à mes critères	Haute
US-04	**Cabinet titulaire**	recevoir des candidatures et les accepter/refuser	gérer efficacement mes remplacements	Haute
US-05	**Infirmier (tous)**	envoyer des messages chiffrés avec pièces jointes	échanger des informations sensibles en toute sécurité	Haute
US-06	**Infirmier (tous)**	envoyer un vocal et obtenir sa transcription automatique	gagner du temps sur mes transmissions	Haute

US-07	**Cabinet titulaire**	visualiser mes remplacements sur un calendrier	anticiper mes besoins de remplacement	Moyenne
US-08	**Infirmier remplaçant**	recevoir une notification quand une tournée correspond à mon profil	ne pas rater d'opportunité	Moyenne
US-09	**Infirmier (tous)**	consulter l'historique de mes missions et statistiques	suivre mon activité professionnelle	Basse
US-10	**Cabinet titulaire**	transmettre les informations patient de manière sécurisée	assurer la continuité des soins en conformité RGPD	Haute

Ces user stories constituent le backlog produit initial (et non un backlog de sprint): elles formalisent, du point de vue utilisateur, l'ensemble des fonctionnalités attendues décrites en section 3.

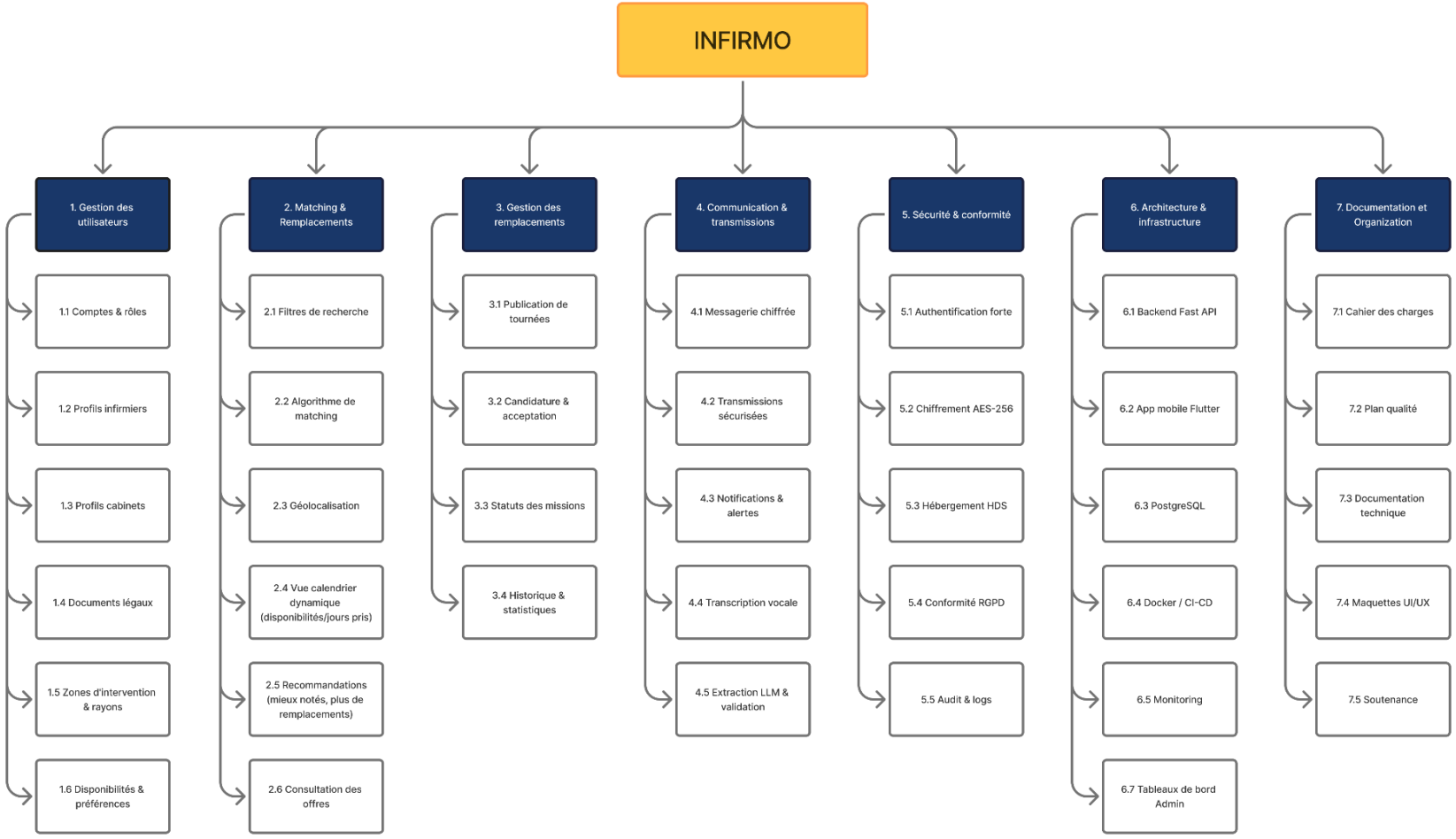
Chaque US est tracée vers un ou plusieurs modules F1–F6 (voir matrice ci-dessous) et vers les tickets du backlog détaillé (section 7.4 « Backlog général consolidé », colonne « Fonctions CDC / US »).

Le backlog de sprint correspond, lui, au sous-ensemble d'US engagé par itération, planifié via les milestones M1 à M6.

Les premières US développées sont celles de priorité Haute (US-01 à US-06, US-10), positionnées sur M1–M2.

4. PBS (Product Breakdown Structure)

Le Product Breakdown Structure (PBS) décompose le produit en lots et sous-lots fonctionnels, du niveau le plus général (modules métier) au niveau le plus détaillé (composants livrables). Il joue le rôle de référentiel partagé pour : (1) structurer le backlog GitHub Projects et la matrice RACI (section 6.2), (2) estimer la charge de chaque lot et alimenter la roadmap (section 7.4), (3) assurer la traçabilité entre les fonctionnalités attendues (section 3) et les livrables techniques (section 16). Chaque feuille du PBS représente une unité de travail planifiable et vérifiable contre la Definition of Done (section 12).


INFIRMO
1. Gestion des utilisateurs
1.1 Comptes & rôles
1.2 Profils infirmiers
1.3 Profils cabinets
1.4 Documents légaux
1.5 Zones d'intervention & rayons
1.6 Disponibilités & préférences
2. Matching & Remplacements
2.1 Filtres de recherche
2.2 Algorithme de matching
2.3 Géolocalisation
2.4 Vue calendrier dynamique (disponibilités/jours pris)
2.5 Recommandations (mieux notés, plus de remplacements)
2.6 Consultation des offres
3. Gestion des remplacements
3.1 Publication de tournées
3.2 Candidature & acceptation
3.3 Statuts des missions
3.4 Historique & statistiques
4. Communication & transmissions
4.1 Messagerie chiffrée
4.2 Transmissions sécurisées
4.3 Notifications & alertes
4.4 Transcription vocale
4.5 Extraction LLM & validation
5. Sécurité & conformité
5.1 Authentification forte
5.2 Chiffrement AES-256
5.3 Hébergement HDS
5.4 Conformité RGPD
5.5 Audit & logs
6. Architecture & infrastructure
6.1 Backend Fast API
6.2 App mobile Flutter
6.3 PostgreSQL
6.4 Docker / CI-CD
6.5 Monitoring
6.7 Tableaux de bord Admin
7. Documentation et Organization
7.1 Cahier des charges
7.2 Plan qualité
7.3 Documentation technique
7.4 Maquettes UI/UX
7.5 Soutenance

5. WBS (Work Breakdown Structure)

Le WBS, ou Work Breakdown Structure, permet de décomposer le projet Infirmo en grands lots de travail. Contrairement au PBS, qui présente les composants et livrables du projet, le WBS décrit les travaux nécessaires pour concevoir, développer, tester, documenter et présenter la solution.

Le WBS du projet Infirmo est organisé en neuf parties : gestion des utilisateurs, recherche et matching, gestion des remplacements, communication, sécurité et conformité, architecture applicative, infrastructure, tests et documentation.

1. Gestion des utilisateurs

1.1 Définir les comptes et les rôles

Définir les différents types d'utilisateurs de l'application : infirmiers titulaires et remplaçants. Préciser les droits associés à chaque rôle afin de sécuriser l'accès aux fonctionnalités.

1.2 Développer l'authentification

Mettre en place les écrans de connexion et d'inscription, gérer les sessions utilisateur et connecter l'authentification à l'API backend.

1.3 Développer les profils infirmiers

Permettre aux infirmiers de créer, consulter et modifier leur profil professionnel, avec les informations nécessaires au matching : identité, compétences, zone d'intervention, disponibilités et préférences.

1.4 Développer les profils cabinets

Permettre aux cabinets de créer, consulter et modifier leur profil, avec les informations nécessaires à la publication des offres de remplacement.

1.5 Gérer les documents légaux

Prévoir l'ajout, l'association et le suivi des documents légaux demandés aux utilisateurs, notamment pour les profils infirmiers.

1.6 Gérer les disponibilités et préférences

Permettre aux infirmiers de renseigner leurs disponibilités, leur rayon d'intervention et leurs préférences de mission afin d'améliorer la pertinence du matching.

2. Recherche, matching et calendrier

2.1 Développer la consultation des offres

Permettre aux infirmiers de consulter les offres de remplacement disponibles, avec les informations essentielles : dates, localisation, type de mission et cabinet concerné.

2.2 Développer les filtres de recherche

Mettre en place des filtres permettant de rechercher les offres selon différents critères : localisation, dates, type de soins, disponibilité ou préférences.

2.3 Intégrer la géolocalisation

Exploiter les données géographiques pour afficher les offres proches de l'utilisateur et faciliter la recherche de missions adaptées à sa zone d'intervention.

2.4 Développer l'algorithme de matching

Définir les critères de correspondance entre les offres des cabinets et les profils des infirmiers. Classer les offres selon leur pertinence en fonction des disponibilités, de la distance et des préférences.

2.5 Développer le calendrier dynamique

Créer une vue calendrier permettant d'afficher les disponibilités, les jours déjà occupés et les missions acceptées. Différencier visuellement les statuts afin de faciliter la lecture du planning.

2.6 Afficher les recommandations

Mettre en avant les offres les plus pertinentes pour l'utilisateur, notamment les missions proches, compatibles avec ses disponibilités ou correspondant à ses préférences.

3. Gestion des remplacements et missions

3.1 Développer la publication de tournées

Permettre aux cabinets de publier une tournée ou une offre de remplacement en indiquant les dates, la localisation, le type de soins et les informations utiles à la mission.

3.2 Développer le parcours de candidature

Permettre aux infirmiers de candidater à une mission et aux cabinets de consulter les candidatures reçues.

3.3 Gérer l'acceptation et le refus des candidatures

Permettre aux cabinets d'accepter ou de refuser une candidature, puis informer l'infirmier du résultat.

3.4 Gérer les statuts des missions

Définir et afficher les différents statuts d'une mission : en attente, acceptée, refusée, en cours ou terminée.

3.5 Développer l'historique des missions

Permettre aux utilisateurs de consulter les missions passées, en cours et à venir afin d'assurer un suivi clair de leur activité.

3.6 Développer les statistiques

Afficher des indicateurs simples liés aux missions, comme le nombre de remplacements effectués, les missions en attente ou les missions terminées.

4. Communication et transmissions

4.1 Développer la messagerie

Créer une messagerie intégrée permettant aux infirmiers et aux cabinets d'échanger directement dans l'application.

4.2 Sécuriser les transmissions

Prévoir un espace dédié aux transmissions sensibles liées aux missions, accessible uniquement aux utilisateurs autorisés.

4.3 Mettre en place les notifications

Notifier les utilisateurs lors d'événements importants : nouvelle offre, candidature reçue, candidature acceptée, nouveau message ou transmission.

4.4 Développer les notes vocales

Permettre l'enregistrement de notes vocales depuis l'application mobile afin de faciliter la saisie d'informations sur le terrain.

4.5 Développer la transcription vocale

Intégrer un service IA capable de transformer une note vocale en texte. Afficher la transcription à l'utilisateur et permettre sa correction avant validation.

4.6 Valider les transmissions

Permettre à l'utilisateur de vérifier, corriger et valider les informations transmises avant leur enregistrement définitif.

5. Sécurité, RGPD et conformité HDS

5.1 Sécuriser l'authentification et les accès

Protéger les comptes utilisateurs et appliquer les permissions en fonction des rôles afin d'éviter les accès non autorisés.

5.2 Protéger les données sensibles

Sécuriser les échanges entre l'application mobile, le backend, la base de données et le service IA. Prévoir le chiffrement des données sensibles lorsque cela est nécessaire.

5.3 Préparer la conformité RGPD

Identifier les données personnelles traitées, définir les règles de conservation, prévoir la suppression ou l'anonymisation des données et documenter les choix liés à la protection des données.

5.4 Préparer l'hébergement HDS

Identifier les contraintes liées à l'hébergement de données de santé, comparer les solutions possibles et préparer une infrastructure compatible avec les exigences HDS.

5.5 Mettre en place l'audit et la traçabilité

Journaliser les accès aux données sensibles et les actions importantes afin de garantir la traçabilité des opérations réalisées dans l'application.

6. Architecture applicative

6.1 Mettre en place l'application mobile Flutter

Initialiser le projet Flutter, structurer l'application, intégrer la charte graphique et connecter les écrans à l'API backend.

6.2 Mettre en place le backend API

Initialiser le backend, définir les endpoints nécessaires aux fonctionnalités, gérer la logique métier et connecter le backend à la base de données.

6.3 Mettre en place le service IA

Créer le service de transcription vocale, prévoir un endpoint de santé, connecter le service au backend et tester la transcription sur des fichiers audio.

6.4 Mettre en place la base de données

Concevoir le modèle de données, configurer PostgreSQL et PostGIS, créer les migrations et tester les requêtes métier et géographiques.

6.5 Définir les échanges entre les composants

Définir les interactions entre l'application mobile, le backend, la base de données, le service IA et les services de notification.

7. Infrastructure, DevOps et déploiement

7.1 Mettre en place les dépôts GitHub

Créer un dépôt GitHub principal pour le projet, organiser les accès de l'équipe et préparer la structure de base : dossiers mobile, backend, ia, infra et docs, README principal, .gitignore et documentation initiale par module.

7.2 Organiser le travail avec Git

Définir une stratégie de branches, protéger les branches principales, mettre en place les conventions de commit et organiser les revues de code.

7.3 Mettre en place Docker

Créer les Dockerfiles nécessaires, configurer le docker-compose du projet et permettre un lancement local simple pour les développeurs.

7.4 Mettre en place l'intégration continue

Créer les workflows de test et de build pour le mobile, le backend et le service IA afin de détecter les erreurs avant intégration.

7.5 Préparer les environnements de déploiement

Préparer les environnements local, staging et production. Configurer le nom de domaine, les certificats SSL et les variables d'environnement.

7.6 Prévoir le déploiement et le rollback

Automatiser le déploiement en staging, préparer le déploiement en production et définir une procédure de retour arrière en cas de problème.

7.7 Mettre en place le monitoring

Collecter les métriques, centraliser les logs, créer des alertes et préparer des tableaux de bord pour suivre l'état de l'application.

8. Tests, qualité et validation

8.1 Définir la stratégie de test

Identifier les tests nécessaires pour valider les fonctionnalités principales, la sécurité, les performances et les intégrations entre les composants.

8.2 Réaliser les tests unitaires

Tester les parties isolées du backend, de l'application mobile et du service IA afin de vérifier le comportement de chaque composant.

8.3 Réaliser les tests d'intégration

Tester les échanges entre le mobile, le backend, la base de données, le service IA et les services externes.

8.4 Réaliser les tests end-to-end

Valider les parcours principaux : inscription, connexion, publication d'une tournée, candidature, acceptation, messagerie et transcription vocale.

8.5 Réaliser les tests de performance

Tester les performances de l'API, du matching géographique et du service de transcription afin d'identifier les points d'optimisation.

8.6 Assurer la qualité du projet

Définir une définition de terminé, suivre les anomalies, corriger les erreurs bloquantes et vérifier la cohérence entre les livrables et les objectifs du projet.

9. Documentation, organisation et promotion

9.1 Rédiger la description du projet

Présenter le besoin auquel répond Infirmo, les utilisateurs cibles, la proposition de valeur et les principales fonctionnalités.

9.2 Maintenir les documents de projet

Présenter le PBS, le WBS, les choix fonctionnels et techniques, ainsi que les principales contraintes du projet.

9.3 Organiser le travail d'équipe

Définir les rôles et responsabilités, préparer les outils de suivi, suivre l'avancement des tâches et fixer les objectifs des prochains jalons.

9.4 Assurer le reporting

Présenter l'avancement du projet, les difficultés rencontrées, les solutions envisagées, les décisions prises et les actions réalisées.

9.5 Préparer la présentation de promotion

Créer un support clair et attractif pour présenter le projet, expliquer le problème traité, valoriser les fonctionnalités principales et montrer l'intérêt technique du projet.

9.6 Préparer les livrables GitHub

Déposer les documents demandés sur GitHub, structurer les fichiers du projet et vérifier la lisibilité des livrables.

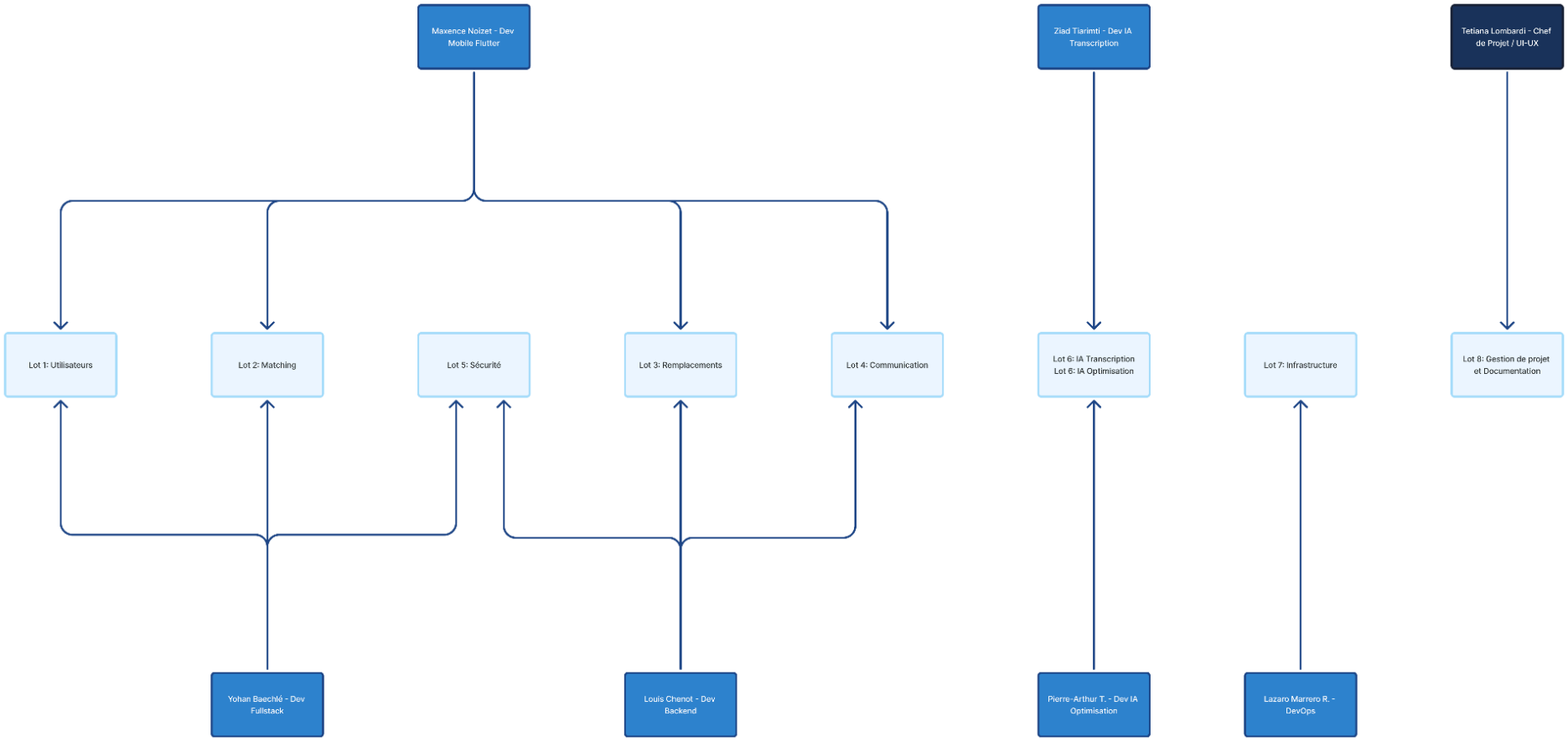
Description des fonctions du WBS

Le projet Infirmo vise à faciliter la mise en relation entre cabinets et infirmiers remplaçants. La gestion des utilisateurs permet de créer des profils adaptés aux deux types d'acteurs, avec leurs documents, disponibilités et zones d'intervention.

La recherche, le matching et le calendrier permettent d'identifier les offres pertinentes selon les critères définis par l'utilisateur. La gestion des remplacements couvre la publication des tournées, les candidatures, l'acceptation des missions et le suivi de leur statut.

La communication intégrée permet aux utilisateurs d'échanger dans l'application et de transmettre des informations liées aux missions. Le service IA apporte une aide à la saisie grâce à la transcription vocale. Les parties sécurité, RGPD et HDS permettent d'encadrer la protection des données sensibles. Enfin, l'architecture technique, l'infrastructure, les tests et la documentation assurent que le projet reste maintenable, testable et présentable dans un cadre professionnel.

6. OBS (Organizational Breakdown Structure)



6.1. Composition de l'équipe

Membre	Poste	Responsabilités
Tetiana Lombardi	Chef de Projet / UI-UX	Coordination Agile, maquettes Figma, rituels Scrumban, planification
Maxence Noizet	Dev Mobile Flutter	Développement app mobile (réseau, messagerie, transmissions)
Yohan Baechlé	Dev Fullstack	APIs, base PostgreSQL, sécurité
Louis Chenot	Dev Backend	Python FastAPIs, WebSocket temps réel
Ziad Tiarimti	Dev Backend	Intégration backend
Pierre-Arthur Toutouom Y.	Dev IA	Transcription Whisper + extraction LLM
Lazaro Marrero Rousse	DevOps	Docker Compose, CI/CD GitHub Actions, monitoring

6.2. Matrice de responsabilités (RACI)

R = Responsable A = Approbateur C = Consulté I = Informé

Lot / Module	TL	MN	YB	LC	PAT	ZT	LMR
1. Gestion des utilisateurs	A	I	R	I	I	I	I
1.1 Définir les comptes et les rôles	A	I	R	C	I	I	I
1.2 Développer l'authentification	A	R	C	C	I	I	I
1.3 Développer les profils infirmiers	A	R	C	I	I	I	I

1.4 Développer les profils cabinets	A	R	C	I	I	I	I
1.5 Gérer les documents légaux	A	I	R	I	I	I	C
1.6 Gérer disponibilités & préférences	A	R	C	I	I	I	I
2. Recherche, matching et calendrier	A	I	I	I	R	C	I
2.1 Consultation des offres	A	R	C	I	I	I	I
2.2 Filtres de recherche	A	R	C	I	I	I	I
2.3 Géolocalisation	A	R	C	C	I	I	I
2.4 Algorithme de matching	A	I	C	I	C	R	I
2.5 Calendrier dynamique	A	R	I	C	I	I	I
2.6 Recommandations	A	I	I	I	R	C	I
3. Gestion des remplacements et missions	A	I	I	R	I	I	I
3.1 Publication de tournées	A	C	R	C	I	I	I
3.2 Parcours de candidature	A	C	C	R	I	I	I
3.3 Acceptation / refus candidatures	A	C	C	R	I	I	I
3.4 Statuts des missions	A	C	I	R	I	I	I
3.5 Historique des missions	A	R	C	I	I	I	I
3.6 Statistiques	A	R	C	C	I	I	I
4. Communication et transmissions	A	R	I	C	I	I	I
4.1 Messagerie	A	R	C	C	I	I	I
4.2 Sécuriser les transmissions	A	C	R	C	I	C	I
4.3 Notifications	A	R	I	C	I	I	I
4.4 Notes vocales	A	R	I	I	C	I	I

4.5 Transcription vocale	A	I	I	I	R	C	I
4.6 Valider les transmissions	A	C	C	I	R	C	I
5. Sécurité, RGPD et conformité HDS	A	I	R	I	I	I	C
5.1 Sécuriser l'authentification & accès	A	I	R	C	I	I	C
5.2 Protéger les données sensibles	A	I	R	C	I	C	C
5.3 Préparer la conformité RGPD	A	I	R	I	I	I	C
5.4 Préparer l'hébergement HDS	A	I	C	I	I	I	R
5.5 Audit et traçabilité	A	I	C	C	I	I	R
6. Architecture applicative	A	I	C	C	I	I	R
6.1 Application mobile Flutter	A	R	C	I	I	I	I
6.2 Backend API	A	I	R	C	I	I	I
6.3 Service IA	A	I	C	I	R	C	I
6.4 Base de données	A	I	R	C	I	I	C
6.5 Échanges entre composants	A	C	R	C	C	C	C
7. Infrastructure, DevOps et déploiement	A	I	I	I	I	I	R
7.1 Mettre en place les dépôts GitHub	A	I	C	C	I	I	R
7.2 Organiser le travail avec Git	A	C	C	C	C	C	R
7.3 Mettre en place Docker	A	I	C	C	I	I	R
7.4 Intégration continue (CI)	A	I	C	C	I	I	R
7.5 Environnements de déploiement	A	I	C	C	I	I	R
7.6 Déploiement et rollback	A	I	C	C	I	I	R
7.7 Monitoring	A	I	I	C	I	I	R

8. Tests, qualité et validation	A	C	R	C	C	C	C
8.1 Définir la stratégie de test	A	C	R	C	C	C	C
8.2 Réaliser les tests unitaires	A	R	C	C	C	C	I
8.3 Réaliser les tests d'intégration	A	C	R	C	C	C	C
8.4 Réaliser les tests end-to-end	A	R	C	C	I	I	C
8.5 Réaliser les tests de performance	A	I	C	C	I	C	R
8.6 Assurer la qualité du projet	R/A	C	C	C	C	C	C
9. Documentation, organisation et promotion	R/A	C	C	C	C	C	C
9.1 Rédiger la description du projet	R/A	C	C	C	C	C	C
9.2 Maintenir les documents de projet	R/A	C	C	C	C	C	C
9.3 Organiser le travail d'équipe	R/A	C	C	C	C	C	C
9.4 Assurer le reporting	R/A	C	C	C	C	C	C
9.5 Préparer la présentation de promotion	R/A	C	C	C	C	C	C
9.6 Préparer les livrables GitHub	A	I	C	C	I	I	R

7. Backlog

7.1. Objectif de la section

Le backlog général du projet Infirmo consolide les tâches issues du cahier des charges et des backlogs individuels transmis par les membres de l'équipe. Il relie chaque ticket à un code WBS afin d'assurer la traçabilité entre le besoin fonctionnel, le travail à réaliser, les responsabilités, les jalons et le suivi opérationnel dans GitHub Projects.

Ce backlog couvre les modules principaux du cahier des charges : gestion des utilisateurs, recherche et matching, gestion des remplacements, communication sécurisée, intelligence artificielle, sécurité et conformité, architecture, infrastructure, tests, documentation et soutenance.

7.2. Hypothèses de charge et de réunions

La charge du projet est cadrée à 21 jours-hommes par membre, réunions incluses. L'équipe étant composée de 7 membres, la charge totale du projet est donc de 147 JH.

L'hypothèse retenue est la suivante : 1 JH correspond à 7 heures de travail. Pour chaque membre, les réunions comprennent 5 réunions longues d'une heure, soit 5 heures, ainsi que des points courts de 15 minutes sur les 16 autres jours de travail, soit 4 heures. Cela représente 9 heures de réunions par personne, soit environ 1,3 JH. La charge productive disponible par membre est donc de 19,7 JH.

Une réserve de capacité d'environ 4,5 JH par personne est volontairement provisionnée sur la phase de finalisation (avril 2027). Elle absorbe les aléas de mise en production (certification HDS, durcissement sécurité, corrections issues des tests de recette) et sécurise la tenue de l'échéance de livraison.

Sigle	Membre	Rôle	Charge productive	Réunions	Cadrage	Réserve finalisation (avril)	Total avec réserve
TL	Tetiana Lombardi	Chef de projet / UI-UX	19.7 JH	1.3 JH	21.0 JH	+4.5 JH	25.5 JH
MN	Maxence Noizet	Dev mobile Flutter	19.7 JH	1.3 JH	21.0 JH	+4.5 JH	25.5 JH
YB	Yohan Baechlé	Dev fullstack	19.7 JH	1.3 JH	21.0 JH	+4.5 JH	25.5 JH
LC	Louis Chenot	Dev backend	19.7 JH	1.3 JH	21.0 JH	+4.5 JH	25.5 JH
ZT	Ziad Tiarimti	Dev backend	19.7 JH	1.3 JH	21.0 JH	+4.5 JH	25.5 JH
PAT	Pierre-Arthur Toutouom Y.	Dev IA	19.7 JH	1.3 JH	21.0 JH	+4.5 JH	25.5 JH
LMR	Lazaro Marrero Rousse	DevOps	19.7 JH	1.3 JH	21.0 JH	+4.5 JH	25.5 JH
Total	**7 membres**	—	**137.9 JH**	**9.1 JH**	**147.0 JH**	**+31.5 JH**	**178.5 JH**

7.3. Traçabilité WBS

Chaque ticket du backlog est rattaché à un ou plusieurs codes WBS. Le WBS indique le lot de travail concerné et permet de vérifier que le backlog couvre bien les éléments du

cahier des charges. Les codes WBS servent également à faciliter l'intégration du backlog dans le document projet et le suivi dans GitHub Projects.

Code WBS	Élément concerné	Charge estimée
0.1	Réunions projet et coordination transversale	9.1 JH
1.2	Développer l'authentification	5.0 JH
1.2 + 1.1	Développer l'authentification / Définir les comptes et les rôles	2.0 JH
1.2 + 5.1	Développer l'authentification / Sécuriser l'authentification et les accès	1.5 JH
1.3 + 1.4	Développer les profils infirmiers / Développer les profils cabinets	3.0 JH
1.5	Gérer les documents légaux	1.5 JH
1.6 + 2.2	Gérer les disponibilités et préférences / Développer les filtres de recherche	2.0 JH
2.1 + 2.2 + 2.3	Développer la consultation des offres / Développer les filtres de recherche / Intégrer la géolocalisation	3.0 JH
2.1 + 2.2 + 2.3 + 2.4	Développer la consultation des offres / Développer les filtres de recherche / Intégrer la géolocalisation / Développer l'algorithme de matching	2.5 JH
2.5	Développer le calendrier dynamique	2.0 JH
3.1	Développer la publication de tournées	2.0 JH
3.1 + 3.2 + 3.3 + 3.4 + 3.5	Développer la publication de tournées / Développer le parcours de candidature / Gérer l'acceptation et le refus des candidatures / Gérer les statuts des missions / Développer l'historique des missions	3.5 JH
3.2 + 3.3	Développer le parcours de candidature / Gérer l'acceptation et le refus des candidatures	2.5 JH
3.4	Gérer les statuts des missions	1.5 JH
4.1	Développer la messagerie	3.0 JH
4.1 + 4.2 + 4.3	Développer la messagerie / Sécuriser les transmissions / Mettre en place les notifications	2.7 JH
4.1 + 6.5	Développer la messagerie / Définir les échanges entre les composants	2.0 JH
4.2 + 4.6	Sécuriser les transmissions / Valider les transmissions	3.0 JH
4.3	Mettre en place les notifications	2.0 JH
4.4 + 4.5	Développer les notes vocales / Développer la transcription vocale	1.5 JH
4.4 + 4.5 + 6.3 + 8.5	Développer les notes vocales / Développer la transcription vocale / Mettre en place le service IA / Réaliser les tests de performance	2.0 JH
5.1	Sécuriser l'authentification et les accès	2.0 JH
5.2 + 5.3	Protéger les données sensibles / Préparer la conformité RGPD	2.5 JH
5.5	Mettre en place l'audit et la traçabilité	2.0 JH
6.1	Mettre en place l'application mobile Flutter	1.0 JH
6.1 + 9.6	Mettre en place l'application mobile Flutter / Préparer les livrables GitHub	5.5 JH
6.2	Mettre en place le backend API	2.0 JH
6.3	Mettre en place le service IA	6.5 JH
6.3 + 4.5	Mettre en place le service IA / Développer la transcription vocale	3.5 JH
6.3 + 4.6	Mettre en place le service IA / Valider les transmissions	7.0 JH
6.4	Mettre en place la base de données	6.0 JH
6.4 + 7.6 + 5.2	Mettre en place la base de données / Prévoir le déploiement et le rollback / Protéger les données sensibles	1.5 JH
6.5 + 4.5 + 4.6	Définir les échanges entre les composants / Développer la transcription vocale / Valider les transmissions	2.0 JH
6.5 + 8.3	Définir les échanges entre les composants / Réaliser les tests d'intégration	4.0 JH
6.5 + 8.3 + 8.4	Définir les échanges entre les composants / Réaliser les tests d'intégration / Réaliser les tests end-to-end	3.0 JH
6.5 + 8.3 + 9.2 + 9.6	Définir les échanges entre les composants / Réaliser les tests d'intégration / Maintenir les documents de projet / Préparer les livrables GitHub	1.0 JH

7.1	Mettre en place les dépôts GitHub	0.5 JH
7.2	Organiser le travail avec Git	2.5 JH
7.3	Mettre en place Docker	2.0 JH
7.4	Mettre en place l'intégration continue	2.8 JH
7.5 + 5.4	Préparer les environnements de déploiement / Préparer l'hébergement HDS	2.0 JH
7.6	Prévoir le déploiement et le rollback	3.0 JH
7.7	Mettre en place le monitoring	2.0 JH
8.3	Réaliser les tests d'intégration	3.9 JH
8.5	Réaliser les tests de performance	2.2 JH
8.6 + 5.3	Assurer la qualité du projet / Préparer la conformité RGPD	2.0 JH
8.6 + 9.3	Assurer la qualité du projet / Organiser le travail d'équipe	0.5 JH
9.2 + 9.6	Maintenir les documents de projet / Préparer les livrables GitHub	6.9 JH
9.2 + 9.6 + 6.3	Maintenir les documents de projet / Préparer les livrables GitHub / Mettre en place le service IA	0.7 JH
9.3 + 9.4	Organiser le travail d'équipe / Assurer le reporting	3.0 JH
9.3 + 9.6	Organiser le travail d'équipe / Préparer les livrables GitHub	2.0 JH
9.5 + 9.6	Préparer la présentation de promotion / Préparer les livrables GitHub	4.2 JH
Total	—	**147 JH**

7.4. Backlog général consolidé

Le tableau ci-dessous présente le backlog général consolidé du projet. Il en donne une vue synthétique : les détails opérationnels complets (descriptions, critères d'acceptation, sous-tâches) sont maintenus dans les issues GitHub Projects.

Ticket	Resp.	JH	Milestone	Priorité	Epic	WBS	Fonctions CDC / US
PM-1 - Finaliser et maintenir le cahier des charges	TL	3.0	M1	Haute	documentation	9.2 + 9.6	
PM-2 - Produire les maquettes UI/UX Figma MVP	TL	5.5	M1	Haute	mobile	6.1 + 9.6	
PM-3 - Configurer GitHub Projects et le backlog général	TL	2.0	M1	Haute	documentation	9.3 + 9.6	
PM-4 - Suivi ScrumBan et reporting projet	TL	3.0	M1-M6	Haute	documentation	9.3 + 9.4	
PM-5 - Plan qualité, risques et Definition of Done	TL	2.0	M1-M6	Haute	sécurité	8.6 + 5.3	
PM-6 - Préparer la soutenance finale et les livrables	TL	4.2	M6	Haute	documentation	9.5 + 9.6	
MOB-1 - Initialisation projet Flutter et intégration Figma	MN	1.0	M2	Haute	mobile	6.1	
MOB-2 - Écrans d'authentification et création de profils	MN	4.0	M2	Haute	auth	1.2	F1.1, F1.6, US-01
MOB-3 - Configuration zones, disponibilités et préférences	MN	2.0	M2	Haute	matching	1.6 + 2.2	F1.5, F2.1
MOB-4 - Découverte des offres, carte et filtres	MN	3.0	M2	Haute	matching	2.1 + 2.2 + 2.3	F2.1, F2.3, F3.2, US-03
MOB-5 - Calendrier dynamique	MN	2.0	M3	Moyenne	matching	2.5	F2.4, US-07
MOB-6 - Cycle de vie d'un remplacement côté mobile	MN	3.5	M3	Haute	remplacements	3.1 + 3.2 + 3.3 + 3.4 + 3.5	F3.1, F3.3, F3.4, F3.5, US-02, US-04, US-09

MOB-7 - Messagerie, notifications et transmissions côté mobile	MN	2.7	M4	Haute	communication	4.1 + 4.2 + 4.3	F4.1, F4.3, F2.5, US-05, US-08, US-10
MOB-8 - Interface vocal et transcription IA	MN	1.5	M4	Haute	ia	4.4 + 4.5	F4.2, F5.1, US-06
YB-1 - Profils infirmiers et cabinets côté backend	YB	3.0	M2	Haute	auth	1.3 + 1.4	F1.5, F1.6
YB-2 - Documents légaux et validation upload backend	YB	1.5	M2	Haute	auth	1.5	F1.2
YB-3 - Publication de tournées côté backend	YB	2.0	M3	Haute	remplacements	3.1	F3.1, US-02
YB-4 - Aide fullstack intégration mobile/backend M2-M4	YB	4.0	M2-M4	Haute	mobile	6.5 + 8.3	
YB-5 - Sécurité auth forte et RBAC	YB	2.0	M2	Haute	sécurité	5.1	F6.1
YB-6 - Support PostgreSQL/PostGIS et requêtes métier	YB	2.0	M2	Haute	backend	6.4	
YB-7 - Tests d'intégration fullstack Auth / Profil / Matching	YB	3.0	M5	Haute	tests	6.5 + 8.3 + 8.4	
YB-8 - Validation performance mobile et API côté intégration	YB	1.2	M5	Haute	tests	8.5	
YB-9 - Documentation technique fullstack	YB	1.0	M6	Moyenne	documentation	6.5 + 8.3 + 9.2 + 9.6	
LC-1 - Architecture API et conventions REST	LC	1.0	M2	Haute	backend	6.2	
LC-2 - JWT, sessions et refresh tokens	LC	1.5	M2	Haute	auth	1.2 + 5.1	F6.1
LC-3 - Workflow candidature et acceptation	LC	2.5	M3	Haute	remplacements	3.2 + 3.3	F3.3, US-04
LC-4 - Statuts des missions et transitions	LC	1.5	M3	Haute	remplacements	3.4	F3.4
LC-5 - Messagerie texte/photo sécurisée backend	LC	3.0	M4	Haute	communication	4.1	F4.1, US-05
LC-6 - Transmissions sécurisées patients	LC	3.0	M4	Haute	communication	4.2 + 4.6	F4.3, US-10
LC-7 - WebSocket temps réel	LC	2.0	M4	Haute	communication	4.1 + 6.5	
LC-8 - Tests API communication/remplacements	LC	2.2	M5	Haute	tests	8.3	
LC-9 - Documentation OpenAPI backend	LC	1.0	M6	Moyenne	documentation	9.2 + 9.6	
LC-10 - Audit logs backend pour accès sensibles	LC	2.0	M4	Haute	securite	5.5	F6.5
ZT-1 - Architecture API partagée et conventions backend	ZT	1.0	M2	Haute	backend	6.2	
ZT-2 - Setup PostgreSQL + PostGIS et migration initiale	ZT	1.0	M2	Haute	backend	6.4	
ZT-3 - Modèles métier remplacements et santé	ZT	3.0	M2	Haute	backend	6.4	
ZT-4 - Inscription et vérification Ordre Infirmiers	ZT	2.0	M2	Haute	auth	1.2 + 1.1	F1.1, F1.3, US-01
ZT-5 - 2FA optionnelle Email/SMS	ZT	1.0	M2	Haute	auth	1.2	F1.4
ZT-6 - Algorithme de matching et filtres avancés	ZT	2.5	M2	Haute	matching	2.1 + 2.2 + 2.3 + 2.4	F2.1, F2.3, F3.2, US-03, F2.2
ZT-7 - Intégration microservice IA dans le backend	ZT	2.0	M4	Haute	ia	6.5 + 4.5 + 4.6	F5.1, F5.2, F5.3, US-06
ZT-8 - Notifications push et APIs externes	ZT	2.0	M3	Moyenne	communication	4.3	F2.5, US-08
ZT-9 - Chiffrement et conformité RGPD backend	ZT	2.5	M5	Haute	securite	5.2 + 5.3	F6.2, F6.4

ZT-10 - Tests backend intégration services	ZT	1.7	M5	Haute	tests	8.3	
ZT-11 - Passation API et documentation intégration	ZT	1.0	M6	Moyenne	documentation	9.2 + 9.6	
IA-01 - Architecture et choix des modèles IA	PAT	2.0	M2	Haute	ia	6.3	F5.1, F5.2, F5.3
IA-02 - Initialisation microservice FastAPI IA	PAT	0.5	M2	Haute	ia	6.3	F5.1, F5.2, F5.3
IA-03 - Dockerfile et requirements IA	PAT	1.0	M2	Haute	ia	6.3	F5.1, F5.2, F5.3
IA-04 - Intégration modèle Whisper	PAT	2.0	M2	Haute	ia	6.3 + 4.5	F5.1, F5.2, F5.3, US-06
IA-05 - Endpoint POST /api/v1/transcribe	PAT	1.5	M2	Haute	ia	6.3 + 4.5	F5.1, F5.2, F5.3, US-06
IA-06 - Optimisation performances transcription	PAT	2.0	M3	Moyenne	ia	4.4 + 4.5 + 6.3 + 8.5	F4.2, F5.1, US-06, F5.2, F5.3
IA-07 - Pipeline RAG avec contexte patient	PAT	2.0	M4	Haute	ia	6.3 + 4.6	F5.1, F5.2, F5.3
IA-08 - Prompt engineering domaine médical	PAT	2.0	M4	Haute	ia	6.3 + 4.6	F5.1, F5.2, F5.3
IA-09 - Endpoint POST /api/v1/extract	PAT	2.0	M4	Haute	ia	6.3 + 4.6	F5.1, F5.2, F5.3
IA-10 - Formatage et validation sortie Pydantic	PAT	1.0	M4	Haute	ia	6.3 + 4.6	F5.1, F5.2, F5.3
IA-11 - Gestion erreurs et logging IA	PAT	0.5	M4	Haute	ia	6.3	F5.1, F5.2, F5.3
IA-12 - Optimisation latence et quantification	PAT	0.8	M5	Moyenne	ia	6.3	F5.1, F5.2, F5.3
IA-13 - Batching et charge concurrente	PAT	1.7	M5	Moyenne	ia	6.3	F5.1, F5.2, F5.3
IA-14 - Documentation OpenAPI et README IA	PAT	0.7	M6	Moyenne	ia	9.2 + 9.6 + 6.3	F5.1, F5.2, F5.3
INF-1 - Organisation GitHub et dépôts	LMR	0.5	M2	Haute	devops	7.1	
INF-2 - Protection branches main/develop	LMR	0.5	M2	Haute	devops	7.2	
INF-3 - CONTRIBUTING et conventions Git	LMR	0.7	M2	Haute	devops	7.2	
INF-4 - Commitlint et conventional commits	LMR	0.5	M2	Haute	devops	7.2	
INF-5 - git-cz commits guidés	LMR	0.3	M2	Moyenne	devops	7.2	
INF-6 - semantic-release minimal	LMR	0.5	M2	Moyenne	devops	7.2	
INF-7 - Templates issues et PR avec DoD	LMR	0.5	M2	Moyenne	devops	8.6 + 9.3	
INF-8 - Docker Compose local	LMR	2.0	M2	Haute	devops	7.3	
INF-9 - CI Backend GitHub Actions	LMR	1.0	M2	Haute	devops	7.4	
INF-10 - CI Mobile Flutter GitHub Actions	LMR	1.0	M2	Haute	devops	7.4	
INF-11 - CI Microservice IA GitHub Actions	LMR	0.8	M2	Haute	devops	7.4	
INF-12 - Staging et POC HDS	LMR	2.0	M2	Haute	securite	7.5 + 5.4	
INF-13 - Déploiement automatique staging	LMR	1.5	M3	Haute	devops	7.6	
INF-14 - Déploiement production et rollback	LMR	1.5	M6	Haute	devops	7.6	

INF-15 - Monitoring Prometheus/Grafana	LMR	2.0	M3	Moyenne	devops	7.7	
INF-16 - Backups PostgreSQL et restauration	LMR	1.5	M5	Haute	devops	6.4 + 7.6 + 5.2	
INF-17 - Tests performance Locust/k6	LMR	1.0	M3	Moyenne	tests	8.5	
INF-18 - README infra et documentation DevOps	LMR	1.9	M6	Haute	documentation	9.2 + 9.6	
MEET-TL - Réunions projet et coordination	TL	1.3	M1-M6	Haute	documentation	0.1	
MEET-MN - Réunions projet et coordination	MN	1.3	M1-M6	Haute	documentation	0.1	
MEET-YB - Réunions projet et coordination	YB	1.3	M1-M6	Haute	documentation	0.1	
MEET-LC - Réunions projet et coordination	LC	1.3	M1-M6	Haute	documentation	0.1	
MEET-ZT - Réunions projet et coordination	ZT	1.3	M1-M6	Haute	documentation	0.1	
MEET-PAT - Réunions projet et coordination	PAT	1.3	M1-M6	Haute	documentation	0.1	
MEET-LMR - Réunions projet et coordination	LMR	1.3	M1-M6	Haute	documentation	0.1	
Total	—	**147.0**	**M1-M6**	—	—	—	—

7.5. Lecture du backlog

La colonne « Resp. » désigne le membre responsable de l'exécution principale du ticket. Les tâches peuvent néanmoins nécessiter des échanges avec d'autres membre conformément à la matrice RACI (section 6.2)

La colonne « Epic » regroupe les tickets par grands domaines fonctionnels ou techniques : authentification, matching, remplacements, communication, IA, backend, DevOps, sécurité, mobile, tests et documentation.

La colonne « Milestone » indique le jalon principal de réalisation. Les milestones M1 à M6 suivent la roadmap du projet : spécifications et setup, MVP core, remplacements, communication et IA, finalisation mobile/IA, puis sécurité, déploiement et soutenance.

La colonne « WBS » garantit la cohérence avec la Work Breakdown Structure (section 5). Un ticket peut être lié à plusieurs codes WBS lorsqu'il couvre une tâche transversale.

7.6. Synthèse de conformité au cadrage projet

- La charge totale respecte le cadrage de 147 JH pour 7 membres.
- Chaque membre dispose de 21 JH au total, dont 19,7 JH productifs et 1,3 JH de réunions.
- Une réserve de finalisation de 4,5 JH/personne (≈ 31,5 JH projet) est provisionnée sur avril 2027 au titre de la gestion de risque, en complément des 147 JH de cadrage.

- Les réunions sont comptabilisées comme charge projet, mais séparées des tickets productifs.
- Les tâches sont alignées avec les modules fonctionnels F1 à F6 et les user stories du cahier des charges.

- Les tickets sont reliés au WBS afin de conserver une traçabilité entre cahier des charges, backlog et GitHub Projects.
- Les tâches hors périmètre MVP ou trop larges ont été réduites ou regroupées afin de rester compatibles avec la capacité de l'équipe.

8. Compétences de l'équipe et justification du recrutement

8.1. Compétences identifiées nécessaires

Compétence	Détail	Membre(s) référent(s)
Gestion de projet Agile	Planification, suivi, rituels Scrum/Scrumban	Tetiana Lombardi
UI/UX Design	Maquettes, prototypage, expérience utilisateur mobile	Tetiana Lombardi
Développement mobile Flutter	Développement cross-platform, Dart, widgets	Maxence Noizet / Yohan Baechlé
Développement Backend	FastAPI, APIs REST, WebSocket	Louis Chenot / Ziad Tiarimti
Base de données	PostgreSQL, modélisation, requêtes optimisées	Louis Chenot / Yohan Baechlé / Tetiana Lombardi / Ziad Tiarimti
Intelligence artificielle	NLP, Whisper, LLM, traitement vocal	Pierre-Arthur Toutouom Y.

DevOps & CI/CD	Docker, GitHub Actions, déploiement	Lazaro Marrero Rousse
Sécurité informatique	Chiffrement, RGPD, HDS, JWT	Tous les membres
Communication & rédaction	Documents, présentations, reporting	Tous les membres

8.2. Justification professionnelle du recrutement

La sélection des membres de l'équipe repose sur leurs compétences techniques, leur expérience préalable et leur adéquation avec les besoins du projet :

- **Tetiana Lombardi** : À l'initiative du projet, elle définit la vision produit et assure la coordination globale. Elle possède également des compétences en design d'interface avec Figma, ce qui permet d'encadrer la conception UX/UI.
- **Maxence Noizet** : Dispose d'une première expérience en développement mobile avec Flutter et Dart. Il intervient principalement sur la réalisation de l'application mobile multiplateforme.
- **Yohan Baechlé** : A développé plusieurs applications en Flutter et possède une capacité à intervenir également sur le backend si nécessaire, ce qui apporte de la flexibilité à l'équipe.
- **Louis Chenot** : Dispose d'une expérience dans la conception et l'implémentation d'API REST. Il possède également des connaissances en Python, ce qui facilite son adaptation à un backend basé sur FastAPI.
- **Pierre-Arthur Toutouom Y.** : Apporte des compétences en machine learning et en optimisation, nécessaires pour la conception et l'amélioration des modèles liés à l'IA.
- **Ziad Tiarimti** : Possède une expérience en Python et un intérêt marqué pour les problématiques liées au Backend, ce qui renforce l'équipe sur cette composante.
- **Lazaro Marrero Rousse** : Dispose de solides compétences en DevOps, notamment sur Docker, GitHub Actions et les pipelines CI/CD, assurant la mise en place et la fiabilité de l'infrastructure technique.

9. Méthodologie de projet

9.1. Choix de la méthodologie: ScrumBan

Le projet Infirmo présente une forte variabilité des tâches (développement mobile, backend, IA, DevOps) et des dépendances entre membres de l'équipe. Les priorités peuvent évoluer rapidement en fonction des contraintes techniques rencontrées.

Le choix de **ScrumBan** permet de conserver un cadre organisationnel inspiré de Scrum (rituels, suivi régulier), tout en intégrant la flexibilité de Kanban, notamment :

- l'adaptation continue des priorités,
- l'ajout ou la modification de tâches en cours de cycle,
- une gestion fluide du backlog.

Cette approche est particulièrement adaptée à un projet en phase initiale, où les besoins ne sont pas totalement stabilisés.

9.2. Rituels Scrumban

Dans le cadre de ScrumBan, le terme "sprint" est utilisé pour désigner des cycles de travail courts, sans contrainte stricte de contenu. Contrairement à Scrum, ces cycles ne sont pas figés : les tâches peuvent être ajustées en fonction des priorités et des blocages.

Rituel	Fréquence	Durée	Lieu	Objectif
Daily stand-up	Quotidien	15 min	Teams	Point rapide : fait, prévu, blocages
Sprint planning	Début sprint	20 min	Teams	Sélection et estimation des tâches
Sprint review	Fin sprint	20 min	Présentiel/Teams	Démonstration des fonctionnalités
Rétrospective	Fin sprint	20 min	Teams	Retour d'expérience et améliorations

9.3. Outil de planification et de gestion

Github Projects est l'unique plateforme de gestion de projet utilisée par toute l'équipe, permettant le suivi des sprints, l'attribution des tâches et la gestion des livrables.

Outil	Usage	Utilisateurs
GitHub Projects	Gestion des tickets, backlog, sprints, Gantt, risques	Toute l'équipe
GitHub	Gestion du code source (Git Flow)	Toute l'équipe
Teams	Communication quotidienne, visios	Toute l'équipe
Figma	Maquettes et prototypage UI/UX	Tetiana Lombardi
Docker Desktop	Environnement de développement local	Toute l'équipe

9.4. Roadmap et planning macro

Le planning macro ci-dessous présente les 6 milestones du projet, alignés avec la structure Github Projects.

Les milestones ne sont pas strictement séquentiels. Certains lots avancent en parallèle, notamment la communication, l'IA et le DevOps. La charge reste néanmoins maîtrisée, car chaque ticket est estimé en JH et chaque membre reste limité à 21 JH, réunions incluses.

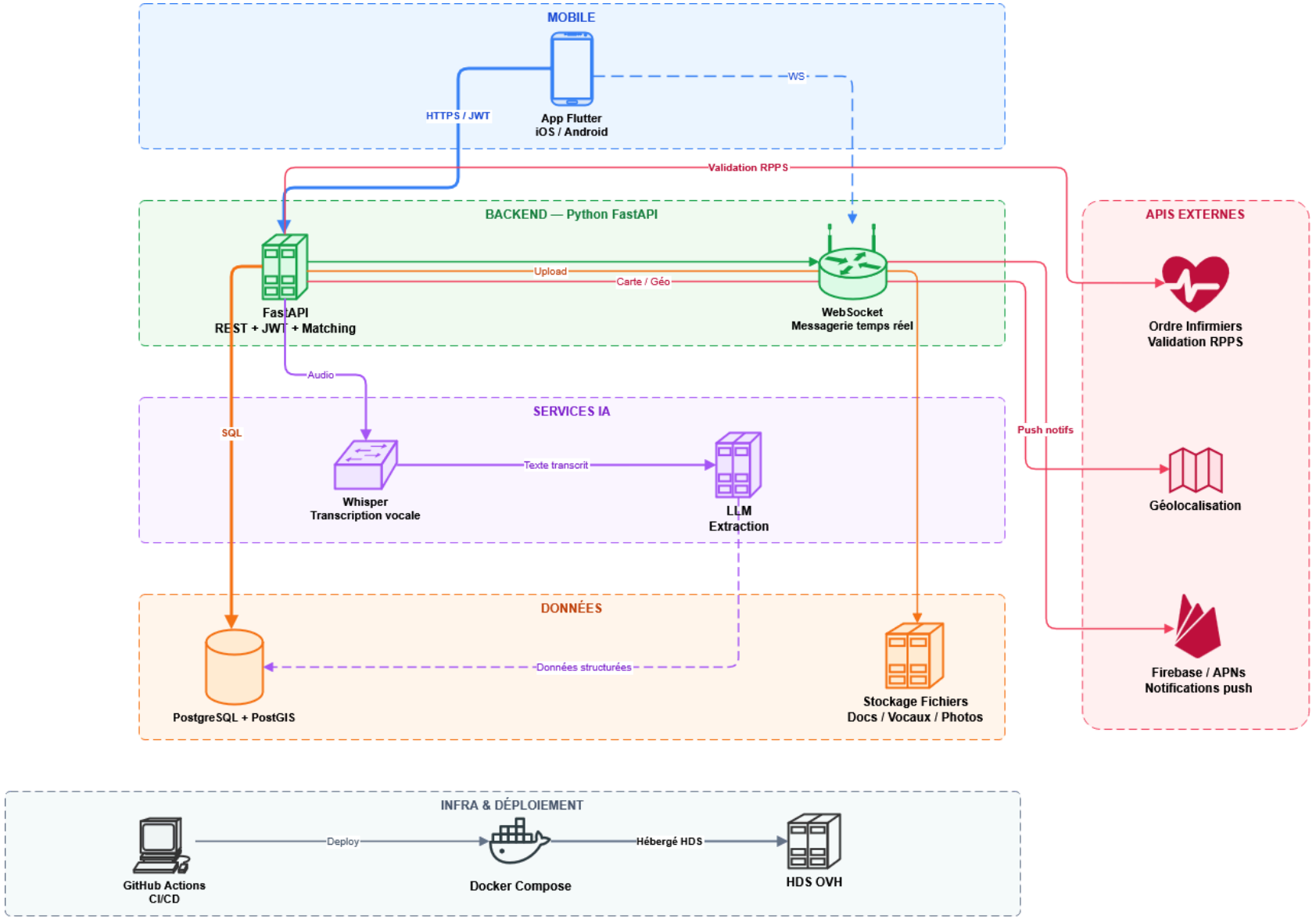
Milestone	Deadline	Contenu	Livrable clé
M1 - Spécifications	Fév - Juin 2026	Cahier des charges, maquettes Figma, schéma BDD, architecture technique, POC, setup env	Docs + maquettes + POC
M2 - MVP Core	Sep - Nov 2026	Authentification, profils infirmier/cabinet, algorithme matching v1	MVP fonctionnel
M3 - Remplacements	Dec 2026	Publication tournées, candidatures, calendrier, notifications email	Module remplacements

M4 - Communication	Oct 2026 - Jan 2027	Messagerie sécurisée, vocaux, transcription IA, notifications push	Module communication
M5 - Mobile & IA	Mar 2027	App mobile Flutter complète, matching IA v2, dashboard statistiques	App mobile + IA
M6 - Sécurité & Deploy	Avr 2027	Infrastructure HDS, conformité RGPD, audit sécurité, doc finale, déploiement prod	Prod + soutenance

10. Choix technologiques et architecture

10.1. Architecture globale

Le diagramme ci-dessous présente l'architecture technique complète du projet Infirmo :


MOBILE
App Flutter
iOS / Android
HTTPS / JWT
WS
Validation RPPS
BACKEND — Python FastAPI
FastAPI
REST + JWT + Matching
WebSocket
Messagerie temps réel
Upload
Carte / Géo
Audio
SQL
SERVICES IA
Whisper
Transcription vocale
Texte transcrit
LLM
Extraction
Push notifs
APIS EXTERNES
Ordre Infirmiers
Validation RPPS
Géolocalisation
Firebase / APNs
Notifications push
DONNÉES
PostgreSQL + PostGIS
Données structurées
Stockage Fichiers
Docs / Vocaux / Photos
INFRA & DÉPLOIEMENT
GitHub Actions
CI/CD
Deploy
Docker Compose
Hébergé HDS
HDS OVH

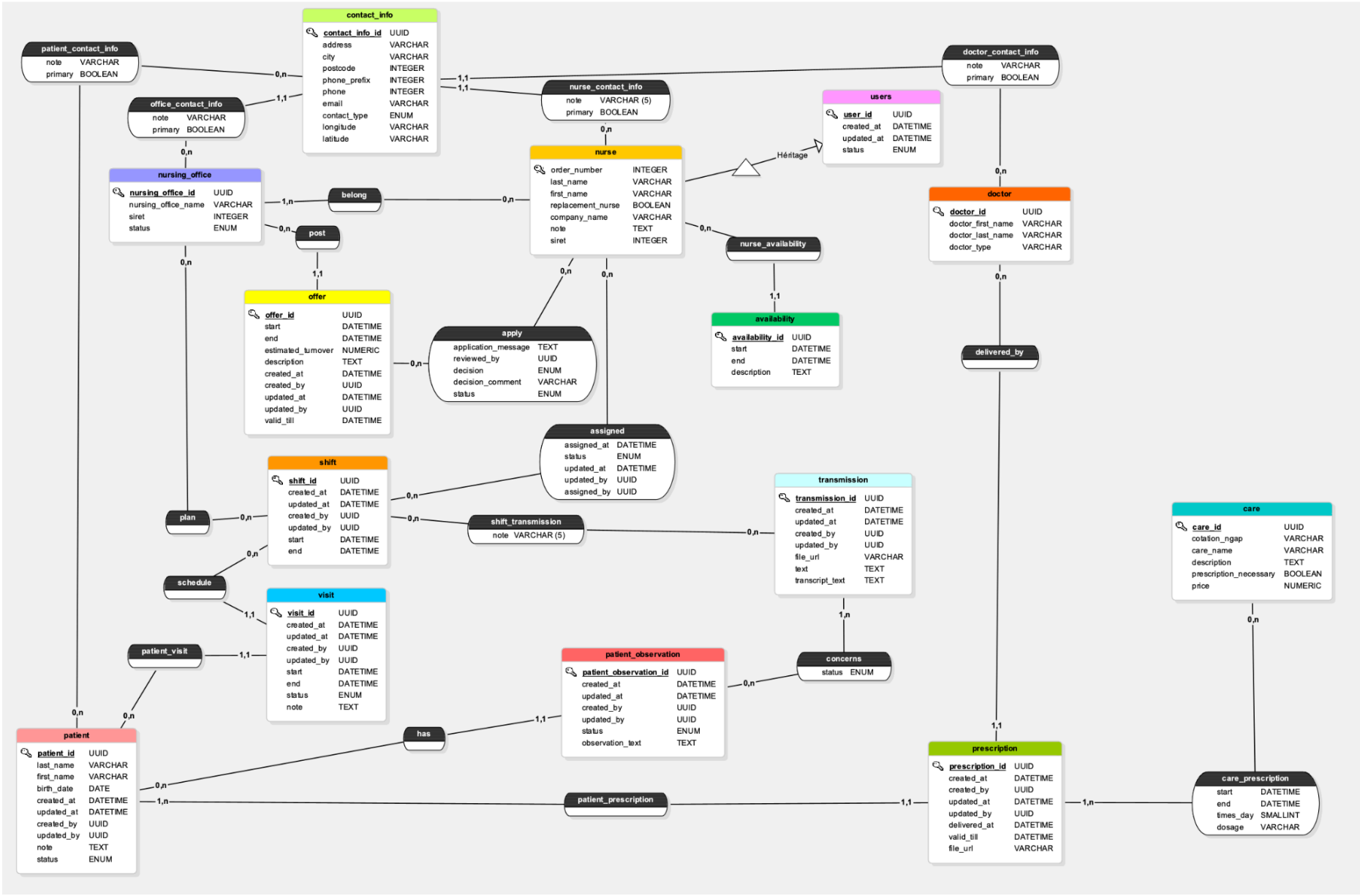
10.2. Modèle de données (ERD)

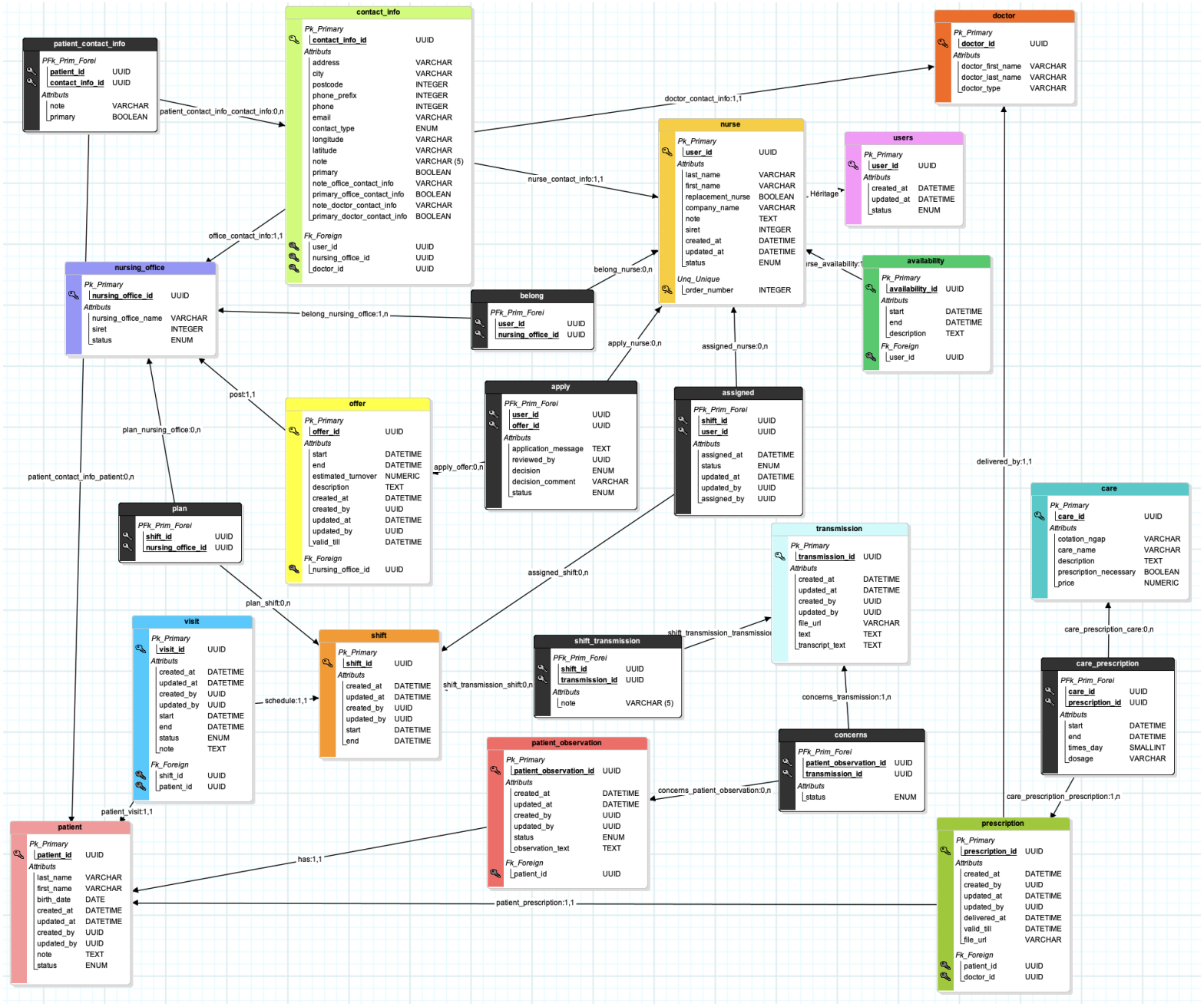
Le modèle de données a été conçu selon la méthode **Merise**, en distinguant les niveaux conceptuel et logique.

Le **Modèle Conceptuel de Données (MCD)** permet de représenter les entités métier ainsi que leurs relations, indépendamment de toute contrainte technique. Il met en évidence les règles de gestion et les cardinalités entre les différentes entités.

Le **Modèle Logique de Données (MLD)** traduit le MCD en une structure relationnelle. Les entités sont transformées en tables, et les relations sont implémentées à l'aide de clés primaires et étrangères.

Les diagrammes présentés ci-dessous illustrent ces deux niveaux de modélisation et permettent de comprendre l'organisation des données au sein de l'application.


contact_info
contact_info_id UUID
address VARCHAR
city VARCHAR
postcode INTEGER
phone_prefix INTEGER
phone INTEGER
email VARCHAR
contact_type ENUM
longitude VARCHAR
latitude VARCHAR
patient_contact_info
note VARCHAR
primary BOOLEAN
office_contact_info
note VARCHAR
primary BOOLEAN
nurse_contact_info
note VARCHAR (5)
primary BOOLEAN
doctor_contact_info
note VARCHAR
primary BOOLEAN
users
user_id UUID
created_at DATETIME
updated_at DATETIME
status ENUM
Héritage
nursing_office
nursing_office_id UUID
nursing_office_name VARCHAR
siret INTEGER
status ENUM
belong
nurse
order_number INTEGER
last_name VARCHAR
first_name VARCHAR
replacement_nurse BOOLEAN
company_name VARCHAR
note TEXT
siret INTEGER
doctor
doctor_id UUID
doctor_first_name VARCHAR
doctor_last_name VARCHAR
doctor_type VARCHAR
post
nurse_availability
offer
offer_id UUID
start DATETIME
end DATETIME
estimated_turnover NUMERIC
description TEXT
created_at DATETIME
created_by UUID
updated_at DATETIME
updated_by UUID
valid_till DATETIME
apply
application_message TEXT
reviewed_by UUID
decision ENUM
decision_comment VARCHAR
status ENUM
availability
availability_id UUID
start DATETIME
end DATETIME
description TEXT
delivered_by
assigned
assigned_at DATETIME
status ENUM
updated_at DATETIME
updated_by UUID
assigned_by UUID
shift
shift_id UUID
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
start DATETIME
end DATETIME
transmission
transmission_id UUID
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
file_url VARCHAR
text TEXT
transcript_text TEXT
care
care_id UUID
cotation_ngap VARCHAR
care_name VARCHAR
description TEXT
prescription_necessary BOOLEAN
price NUMERIC
plan
shift_transmission
note VARCHAR (5)
schedule
visit
visit_id UUID
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
start DATETIME
end DATETIME
status ENUM
note TEXT
patient_visit
patient_observation
patient_observation_id UUID
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
status ENUM
observation_text TEXT
concerns
status ENUM
patient
patient_id UUID
last_name VARCHAR
first_name VARCHAR
birth_date DATE
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
note TEXT
status ENUM
has
prescription
prescription_id UUID
created_at DATETIME
created_by UUID
updated_at DATETIME
updated_by UUID
delivered_at DATETIME
valid_till DATETIME
file_url VARCHAR
patient_prescription
care_prescription
start DATETIME
end DATETIME
times_day SMALLINT
dosage VARCHAR


patient_contact_info
PFk_Prim_Forei
patient_id UUID
contact_info_id UUID
Attributs
note VARCHAR
primary BOOLEAN
patient_contact_info_contact_info:0,n
contact_info
Pk_Primary
contact_info_id UUID
Attributs
address VARCHAR
city VARCHAR
postcode INTEGER
phone_prefix INTEGER
phone INTEGER
email VARCHAR
contact_type ENUM
longitude VARCHAR
latitude VARCHAR
note VARCHAR (5)
primary BOOLEAN
note_office_contact_info VARCHAR
primary_office_contact_info BOOLEAN
note_doctor_contact_info VARCHAR
primary_doctor_contact_info BOOLEAN
Fk_Foreign
user_id UUID
nursing_office_id UUID
doctor_id UUID
doctor_contact_info:1,1
doctor
Pk_Primary
doctor_id UUID
Attributs
doctor_first_name VARCHAR
doctor_last_name VARCHAR
doctor_type VARCHAR
nurse_contact_info:1,1
nurse
Pk_Primary
user_id UUID
Attributs
last_name VARCHAR
first_name VARCHAR
replacement_nurse BOOLEAN
company_name VARCHAR
note TEXT
siret INTEGER
created_at DATETIME
updated_at DATETIME
status ENUM
Unq_Unique
order_number INTEGER
Héritage
users
Pk_Primary
user_id UUID
Attributs
created_at DATETIME
updated_at DATETIME
status ENUM
office_contact_info:1,1
nursing_office
Pk_Primary
nursing_office_id UUID
Attributs
nursing_office_name VARCHAR
siret INTEGER
status ENUM
belong_nurse:0,n
belong
PFk_Prim_Forei
user_id UUID
nursing_office_id UUID
belong_nursing_office:1,n
availability
Pk_Primary
availability_id UUID
Attributs
start DATETIME
end DATETIME
description TEXT
Fk_Foreign
user_id UUID
apply_nurse:0,n
assigned_nurse:0,n
post:1,1
offer
Pk_Primary
offer_id UUID
Attributs
start DATETIME
end DATETIME
estimated_turnover NUMERIC
description TEXT
created_at DATETIME
created_by UUID
updated_at DATETIME
updated_by UUID
valid_till DATETIME
Fk_Foreign
nursing_office_id UUID
apply
PFk_Prim_Forei
user_id UUID
offer_id UUID
Attributs
application_message TEXT
reviewed_by UUID
decision ENUM
decision_comment VARCHAR
status ENUM
apply_offer:0,n
assigned
PFk_Prim_Forei
shift_id UUID
user_id UUID
Attributs
assigned_at DATETIME
status ENUM
updated_at DATETIME
updated_by UUID
assigned_by UUID
delivered_by:1,1
plan_nursing_office:0,n
patient_contact_info_patient:0,n
plan
PFk_Prim_Forei
shift_id UUID
nursing_office_id UUID
plan_shift:0,n
assigned_shift:0,n
transmission
Pk_Primary
transmission_id UUID
Attributs
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
file_url VARCHAR
text TEXT
transcript_text TEXT
care
Pk_Primary
care_id UUID
Attributs
cotation_ngap VARCHAR
care_name VARCHAR
description TEXT
prescription_necessary BOOLEAN
price NUMERIC
care_prescription_care:0,n
visit
Pk_Primary
visit_id UUID
Attributs
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
start DATETIME
end DATETIME
status ENUM
note TEXT
Fk_Foreign
shift_id UUID
patient_id UUID
schedule:1,1
shift
Pk_Primary
shift_id UUID
Attributs
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
start DATETIME
end DATETIME
shift_transmission_shift:0,n
shift_transmission
PFk_Prim_Forei
shift_id UUID
transmission_id UUID
Attributs
note VARCHAR (5)
shift_transmission_transmissio
concerns_transmission:1,n
care_prescription
PFk_Prim_Forei
care_id UUID
prescription_id UUID
Attributs
start DATETIME
end DATETIME
times_day SMALLINT
dosage VARCHAR
patient_observation
Pk_Primary
patient_observation_id UUID
Attributs
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
status ENUM
observation_text TEXT
Fk_Foreign
patient_id UUID
concerns_patient_observation:0,n
concerns
PFk_Prim_Forei
patient_observation_id UUID
transmission_id UUID
Attributs
status ENUM
care_prescription_prescription:1,n
patient_visit:1,1
patient
Pk_Primary
patient_id UUID
Attributs
last_name VARCHAR
first_name VARCHAR
birth_date DATE
created_at DATETIME
updated_at DATETIME
created_by UUID
updated_by UUID
note TEXT
status ENUM
has:1,1
patient_prescription:1,1
prescription
Pk_Primary
prescription_id UUID
Attributs
created_at DATETIME
created_by UUID
updated_at DATETIME
updated_by UUID
delivered_at DATETIME
valid_till DATETIME
file_url VARCHAR
Fk_Foreign
patient_id UUID
doctor_id UUID

10.3. Stack technique

Couche	Technologie	Justification
Mobile	**Flutter**	Cross-platform, performance native, hot reload, large communauté
Backend	**FastAPI**	Léger, performant, rapide à développer, très adapté aux API REST et à l'intégration avec les services IA Python.
Base de données	**PostgreSQL**	Relationnelle, performante, support géospatial (PostGIS)
Temps réel	**WebSocket + Redis pub/sub**	Connexions temps réel natives via FastAPI (WebSocket), Redis pub/sub pour la diffusion des messages et notifications entre instances backend (scalabilité horizontale).
IA Transcription	**Whisper**	Modèle open-source, exécution locale pour confidentialité
Conteneurisation	**Docker**	Environnements reproductibles, isolation des services
CI/CD	**GitHub Actions**	Automatisation build, tests, déploiement
Maquettes	**Figma**	Prototypage collaboratif, design system

10.4. Stratégie de tests

Cette stratégie combine des tests automatisés et des vérifications manuelles afin de couvrir les différents niveaux du projet : backend, base de données, application mobile, service IA et sécurité. L'objectif est de détecter les erreurs le plus tôt possible, de limiter les régressions et de garantir que les fonctionnalités principales restent stables au fil du développement.

Type de test	Outils	Couverture
Tests unitaires backend	pytest	Fonctions, services et règles métier isolées
Tests API / intégration	pytest + FastAPI TestClient / HTTPX	Endpoints, validation des données, communication avec la base PostgreSQL
Tests base de données	pytest + base de test PostgreSQL	Requêtes, contraintes, relations, données cohérentes
Tests mobile	flutter_test	Widgets, logique métier mobile, affichage
Tests E2E	Flutter integration_test	Parcours utilisateur complets entre l'application mobile et le backend
Tests IA	pytest + jeux de données de référence	Qualité de transcription, exactitude des résultats, non-régression
Tests de sécurité	Bandit, audits manuels	Vulnérabilités API, dépendances sensibles, fuites de données
Qualité du code	Ruff, Black, flutter analyze	Linting, formatage, cohérence du code
Tests manuels exploratoires	Postman	Vérification ponctuelle des endpoints et scénarios non automatisés

10.5. Indicateurs de performance (KPI)

Les indicateurs suivants établissent les seuils d'acceptation mesurables pour chaque aspect critique du projet : Ces KPI sont d'abord des éléments techniques servant l'équipe de développement pour piloter la qualité et la performance du produit.

KPI	Seuil d'acceptation	Méthode de mesure
Temps de chargement app	< 2 secondes	Test sur devices réels (Android/iOS)
Temps de réponse API (p95)	< 500 ms	Locust / k6
Disponibilité service	> 99.5%	Monitoring Uptime (staging)
Couverture tests unitaires	70 % sur le cœur métier backend ; tests présents sur mobile et IA pytest-cov / flutter_test	pytest / flutter_test
Transcription vocale	Précision > 90%	Jeu de données de test Whisper
Temps de matching	< 3 secondes	Mesure sur base simulée
Conformité RGPD	100% critères	Checklist audit RGPD
Délai pipeline CI/CD	< 15 minutes	GitHub Actions

11. Plan de qualité

Le plan de qualité établit les normes partagées, les procédures et les dispositifs instaurés pour assurer la performance, la sécurité et la facilité de maintenance du produit fourni.

11.1. Processus de développement

- Méthode : Scrumban avec sprints de 5JH / sprint.
- Gestion du code : GitHub avec Git Flow (main = production, develop = intégration, feature/* = développement).
- Branches protégées : main et develop nécessitent une Pull Request validée.
- Revues de code : PR obligatoires avec au minimum un reviewer. Linting automatique à chaque push.

11.2. Normes et bonnes pratiques

- Conventions de nommage : camelCase (Dart), lower_snake_case (base de données).
- Formatage automatique : Black + Ruff (Python), dart format / flutter analyze (Dart)
- Documentation : README par module, documentation API (Swagger/OpenAPI).
- Sécurité : Aucun secret en clair dans le code (variables d'environnement), validation systématique des entrées.
- Gestion des versions : Tags sémantiques vX.Y.Z sur main après validation staging.

11.3. Conventions Git

- Les branches doivent suivre le format :

  feature/<desc>, bugfix/<desc>, hotfix/<desc>, docs/<desc>, ci/<desc>, test/<desc>, refactor/<desc> ou dev/<desc>.

  Ce format est vérifié automatiquement par les scripts du projet et la CI.

- Les commits doivent respecter la convention **Conventional Commits** :

  feat, fix, docs, style, refactor, test, chore, ci, perf, build.

  Exemple : feat(auth): add JWT cookie login.

- Pour être guidé lors de la rédaction d'un commit, utiliser : npx git-cz.
- Les releases sont générées automatiquement avec semantic-release via la commande npx semantic-release, selon la configuration définie dans release.config.js.

11.4. Gestion de la configuration

Outil de configuration : GitHub, organisé autour d'un dépôt principal structuré par modules : mobile, backend, IA, infrastructure et documentation. Cette organisation

conserve une séparation claire des responsabilités tout en simplifiant la configuration Docker Compose, l'intégration continue et le suivi des livrables.

Le choix du mono-repo vise à simplifier le lancement local via Docker Compose, la coordination DevOps et la gestion des dépendances entre le backend, le service IA, l'application mobile et l'infrastructure.

Stratégie de branches : Git Flow avec branches feature, develop, main. Merge : Squash merge sur develop, merge commit sur main.

11.5. Gestion des tickets

Outil : GitHub Projects. Workflow : To Do → In Progress → In Review → Done. Labels : bug, feature, improvement, documentation, urgent. Attribution : Chaque ticket est assigné à un responsable et lié à un sprint.

11.6. Gestion des livrables et déploiements

Environnements : Local (Docker Compose), Staging (déploiement automatique), Production démo (si HDS possible). Pipeline : Push sur develop → build + tests + déploiement staging. Validation DoD (critères détaillés en section 12) → merge main + tag → prod. CI/CD : GitHub Actions.

11.7. Rôles et prérogatives

Chaque membre a un rôle défini (voir section 6 : OBS) tout en conservant la possibilité de contribuer transversalement. Le chef de projet coordonne les sprints, priorise le backlog et s'assure du respect des échéances.

11.8. Onboarding et ramp-up

Pour faciliter l'intégration de tout nouveau membre ou la prise en main du projet :

- Un README principal est maintenu à la racine du dépôt, complété par des README spécifiques dans chaque module lorsque nécessaire.
- Les conventions de code et les bonnes pratiques sont documentées dans un fichier CONTRIBUTING.md.
- Un environnement Docker prêt à l'emploi permet de lancer le projet en une commande.
- Les maquettes Figma servent de référence visuelle pour comprendre rapidement les fonctionnalités.

12. Definition of Done (DoD)

La Definition of Done établit les critères partagés et objectifs qu'une fonctionnalité doit remplir pour être considérée comme terminée.

Critère	Détail
Code implémenté et relu	Le code est développé, a passé une revue de code (PR) et est fusionné sur la branche d'intégration.
Critères d'acceptation validés	Les critères définis dans le ticket sont couverts et validés (tests et/ou démonstration).
Tests automatisés OK	Les tests unitaires et d'intégration passent. Les tests E2E passent si applicable.
Pas de régression	Aucune régression critique n'est introduite (build CI vert).
Sécurité respectée	Authentification, autorisations et absence de secrets en clair sont vérifiés.
Documentation à jour	README, endpoints API et notes de release sont mis à jour.
Déployé en staging	La fonctionnalité est déployée en environnement de recette/staging et validée.

13. Analyse des risques

REGISTRE DES RISQUES						
ID	**Risque**	**Probabilité**	**Impact**	**Criticité**	**Niveau**	**Plan de mitigation**
R1	Indisponibilité d'un membre	Moyenne (3)	Élevé (4)	**12**	**Élevée**	Documentation complète, code reviews croisées, onboarding facilité pour redistribuer les tâches.
R2	Complexité hébergement HDS	Élevée (5)	Élevé (4)	**20**	**Critique**	Anticiper la certification dès M1, POC avec hébergeur HDS en M2, budget de contingence

						prévu.
R3	Complexité technique IA/ML (Whisper local)	Moyenne (3)	Moyen (3)	**9**	**Élevée**	POC technique dès M1, fallback vers API cloud si performance locale insuffisante.
R4	Scope creep (dérive du périmètre)	Élevée (5)	Élevé (4)	**20**	**Critique**	Backlog priorisé, périmètre MVP verrouillé, arbitrage par la chef de projet à chaque sprint.
R5	Dépendances APIs externes (Ordre Infirmiers, Google Maps)	Moyenne (3)	Moyen (3)	**9**	**Élevée**	Mock des APIs en développement, tests d'intégration dédiés, monitoring des quotas.
R6	Retard sur les livrables	Moyenne (3)	Élevé (4)	**12**	**Élevée**	Sprints courts (5 JH), suivi quotidien (daily), alertes automatiques GitHub Projects.
R7	Fuite de données sensibles	Faible (1)	Critique (5)	**5**	**Modérée**	Chiffrement E2E, audit OWASP, revue sécurité à chaque sprint, aucun secret en clair.

MATRICE DES RISQUES

Probabilité × Impact — Criticité = P × I

IMPACT ↑	PROBABILITÉ →						
		Faible (1)	**Peu probable (2)**	**Moyenne (3)**	**Probable (4)**	**Élevée (5)**	**LÉGENDE CRITICITÉ**
	Critique (5)	**5 R7**	10	15	20	25	≥ 15 Critique
	Élevé (4)	4	8	**12 R1 R6**	16	**20 R2 R4**	9–14 Élevée
	Moyen (3)	3	6	**9 R3**	12	15	4–8 Modérée

				R5			
	Faible (2)	2	4	6	8	10	1–3 Faible
	Négligeable (1)	1	2	3	4	5	R1=Indisponibilité R2=HDS R3=IA/ML R4=Scope creep R5=APIs R6=Retard R7=Fuite

14. Ressources matérielles et estimation des coûts

Ressource	Spécifications	Durée	Coût estimé
Support Business OVH (obligatoire HDS)	Requis contractuellement pour activer la certification HDS	Mensuel	**~250 €/mois**
Serveur IA	Serveur GPU dédié pour exécution **locale** de Whisper + LLM (transcription vocale + extraction d'informations), choix local imposé par la confidentialité des données de santé. GPU NVIDIA (VRAM adaptée au modèle retenu), CPU multi-cœurs, 32–64 Go RAM, SSD NVMe.	Achat unique (CAPEX)	**~5000 €**
Hébergement HDS Production (total)	Instance Public Cloud certifiée HDS (OVH).	Annuel	**6500 €/an**
Nom de domaine	infirmo.fr ou similaire	Annuel	**~50 €/an**
Certificat SSL	Let's Encrypt	Gratuit	**0 €**

Lancement commercial Grand Est	Publicité sur différents réseaux sociaux	Mensuel	**1000 €/mois**
Juridique & conformité	Validation niveau légal des documents du projet	Achat unique	**8000 €**

15. Contraintes techniques et réglementaires

15.1. Architecture

- Backend léger et performant basé sur **Fast API**, facilitant le développement rapide d'API REST et l'intégration avec les services IA.
- API sécurisée via **HTTPS**, authentification par **JWT** et validation stricte des entrées.
- Base de données relationnelle **PostgreSQL** avec extension **PostGIS** pour la gestion des données géospatiales.

15.2. Sécurité et conformité

- Respect strict du RGPD (consentement, droit à l'oubli, portabilité).
- Protection des données (chiffrement AES-256 au repos, TLS en transit).
- Gestion fine des droits d'accès (RBAC — Role-Based Access Control).
- Hébergement conforme HDS pour les données de santé.

15.3. Performance

Les seuils de performance sont définis dans la section 10.5 (KPI). Les exigences principales sont un temps de chargement inférieur à 2 secondes sur mobile, un temps de réponse API inférieur à 500 ms (p95) et un support du mode hors-ligne partiel (cache local).

15.4. Données patients (RGPD)

- Données de test anonymisées en développement.

15.5. Comment la conformité RGPD sera vérifiée

Un premier point en décembre 2026, lors du prochain rendez-vous de suivi, sur ce qui se prépare dès la conception : le registre des données
que nous collectons, le choix de ne garder que le strict nécessaire, les écrans de consentement et le choix d'un hébergeur certifié pour les données de santé. Ces éléments sont présentés à partir de nos documents de travail.

Un second point en avril 2027, à la livraison de l'application, où l'on montre le fonctionnement réel : la suppression des données après 30 jours, la possibilité pour un utilisateur de supprimer son compte et ses données, et le chiffrement des échanges. Ces points sont démontrés directement dans l'application.

Comme nos documents et notre code sont suivis dans Git, chaque avancée est datée et consultable : on peut vérifier ce qui a réellement été fait, et quand.
Si un point n'est pas prêt à l'échéance, nous l'indiquons et donnons la date à laquelle il le sera.

16. Livrables attendus

Cette section résume les livrables et les échéances. En complément de la roadmap détaillée (section 9.4), elle doit être lue : chaque livrable ci-dessous est lié à un milestone (M1 à M6) en précisant le contenu, les fonctionnalités couvertes et le critère de validation.

Les maquettes UI/UX du MVP sont réalisées dans Figma. Elles sont accessibles via un lien partagé depuis le dépôt GitHub du projet, dans le dossier docs/livrables/maquettes/.

Le fichier Figma constitue la référence visuelle pour la conception de l'application mobile et sert de support aux échanges de conception, à la validation des parcours utilisateurs et à l'implémentation progressive des écrans Flutter.

Livrable	Échéance	Format
Cahier des charges complet	Juin 2026	Document (.pdf)
Maquettes UI/UX (Figma)	Juin 2026	Figma
Présentation orale (documentation)	Juin 2026	Slides + Documentation
Application mobile (MVP)	Avril 2027	APK / IPA
Backend API déployé	Avril 2027	Serveur
Module IA (transcription)	Avril 2027	Service
Documentation technique	Avril 2027	Document
Présentation orale	Avril 2027	Slides + Démo

17. Évolutions possibles

Les évolutions citées ci-dessous ne font pas partie du périmètre du projet livré en avril 2027. Il s'agit de fonctionnalités jugées pertinentes mais non prioritaires pour la première version, à développer dans une phase ultérieure (post-soutenance) en fonction des retours des premiers utilisateurs et de la viabilité commerciale du produit.

Ces évolutions dans le cadre du projet T-ESP-800 ne donnent lieu à aucune charge de développement.

- Tableau de bord avancé : Dashboard personnalisé avec statistiques d'activité.
- Statistiques d'activité : Analyse des tendances, rapports périodiques.
- Automatisation avancée : Suggestions automatiques de remplacements basées sur l'historique.
- Interconnexion outils externes : Intégration avec logiciels de facturation, agendas, outils métiers.
- Optimisation de tournées IA : Calcul d'itinéraires optimisés en fonction des patients et contraintes horaires.
- Application web : Version web complémentaire pour l'administration des cabinets.

18. Glossaire

Terme	Définition
API	Application Programming Interface Interface de programmation permettant à deux logiciels de communiquer. Dans Infirmo, les API REST exposent les fonctionnalités du backend à l'app mobile Flutter.
CI/CD	Continuous Integration / Continuous Deployment Pratiques d'automatisation qui consistent à intégrer (build, tests) puis déployer le code en continu à chaque modification, garantissant fiabilité et rapidité de mise en production.
GitHub Projects	Outil de gestion de projet collaboratif utilisé par l'équipe
DoD	Definition of Done Liste de critères objectifs (code relu, tests verts, doc à jour, déployé en staging…) qui doivent être satisfaits pour qu'une fonctionnalité soit considérée comme terminée. Détaillée en section 12.

ERD	Entity-Relationship Diagram Diagramme entité-relation modélisant les tables de la base de données, leurs attributs et les associations qui les relient. Voir section 10.2.
Flutter	Framework open-source de Google pour le développement mobile cross-platform
Git Flow	Modèle de branchement Git structuré (main, develop, feature)
HDS	Hébergeur de Données de Santé Certification française obligatoire pour tout hébergement de données de santé à caractère personnel en France. Encadre la sécurité physique, logique et organisationnelle de l'hébergement.
JWT	JSON Web Token Standard d'authentification stateless reposant sur des jetons signés transmis entre client et serveur. Utilisé dans Infirmo pour authentifier les requêtes API après login.
KPI	Key Performance Indicator Indicateur clé de performance permettant de mesurer objectivement l'atteinte d'un objectif. Les KPI techniques du projet sont listés en section 10.5.
LLM	Large Language Model Modèle de langage de grande taille entraîné sur de vastes corpus de texte, capable de générer ou résumer du langage naturel. Utilisé dans Infirmo pour extraire et structurer les informations issues des transmissions vocales.
MVP	Minimum Viable Product Produit minimum viable : version réduite du produit, contenant uniquement les fonctionnalités essentielles, livrée pour valider la valeur perçue auprès des premiers utilisateurs.

OBS	Organizational Breakdown Structure Structure organisationnelle du projet, qui décrit la hiérarchie des rôles et des équipes ainsi que la répartition des responsabilités. Voir section 6.
PR	Pull Request Demande de fusion de code soumise sur la plateforme Git (GitHub) pour faire relire et valider une branche avant son intégration sur develop ou main.
RACI	Responsible, Accountable, Consulted, Informed Matrice qui clarifie, pour chaque tâche, qui réalise (R), qui valide (A), qui est consulté (C) et qui est informé (I). Voir section 6.2.
RGPD	Règlement Général sur la Protection des Données (UE 2016/679), texte européen qui encadre la collecte et le traitement des données personnelles. Impose notamment consentement, minimisation, droit à l'oubli et portabilité.
Scrumban	Méthodologie hybride combinant Scrum et Kanban
WBS	Work Breakdown Structure Décomposition hiérarchique du travail à réaliser pour mener le projet à bien, organisée en lots et tâches livrables (équivalent organisationnel du PBS côté production).
WebSocket	Protocole de communication bidirectionnelle en temps réel
Whisper	Modèle open-source d'OpenAI pour la transcription vocale